Docker

Assignment Document

Assignments 1 – 5

Name: Rahul Kumar Pandey
Email: rahulpandeyyy@gmail.com
Mobile: 7257000333

Assignment 1 – Pull Ubuntu Container & Install Apache2

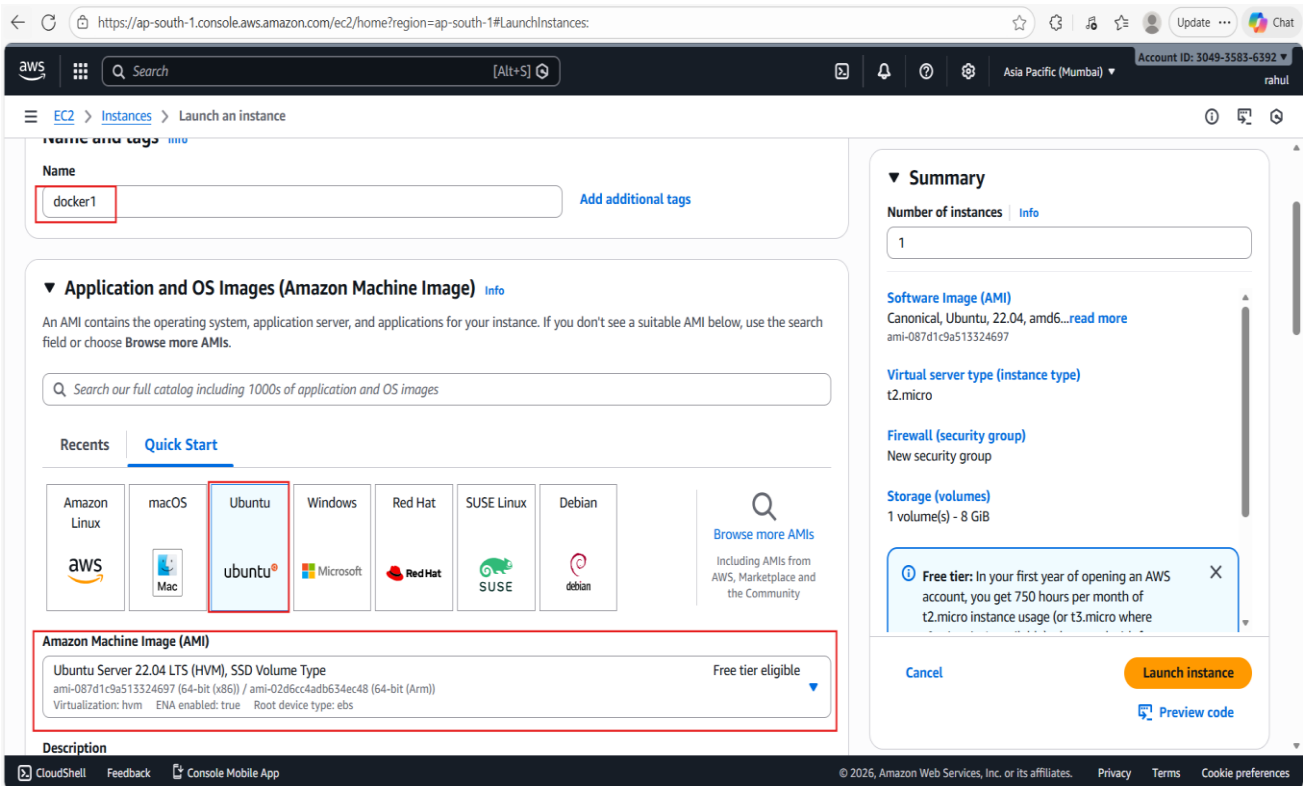
Tasks To Be Performed:

1. Pull Ubuntu container
2. Run this container and map port 80 on the local machine
3. Install Apache2 on this container
4. Check if you are able to access the Apache page on your browser

---

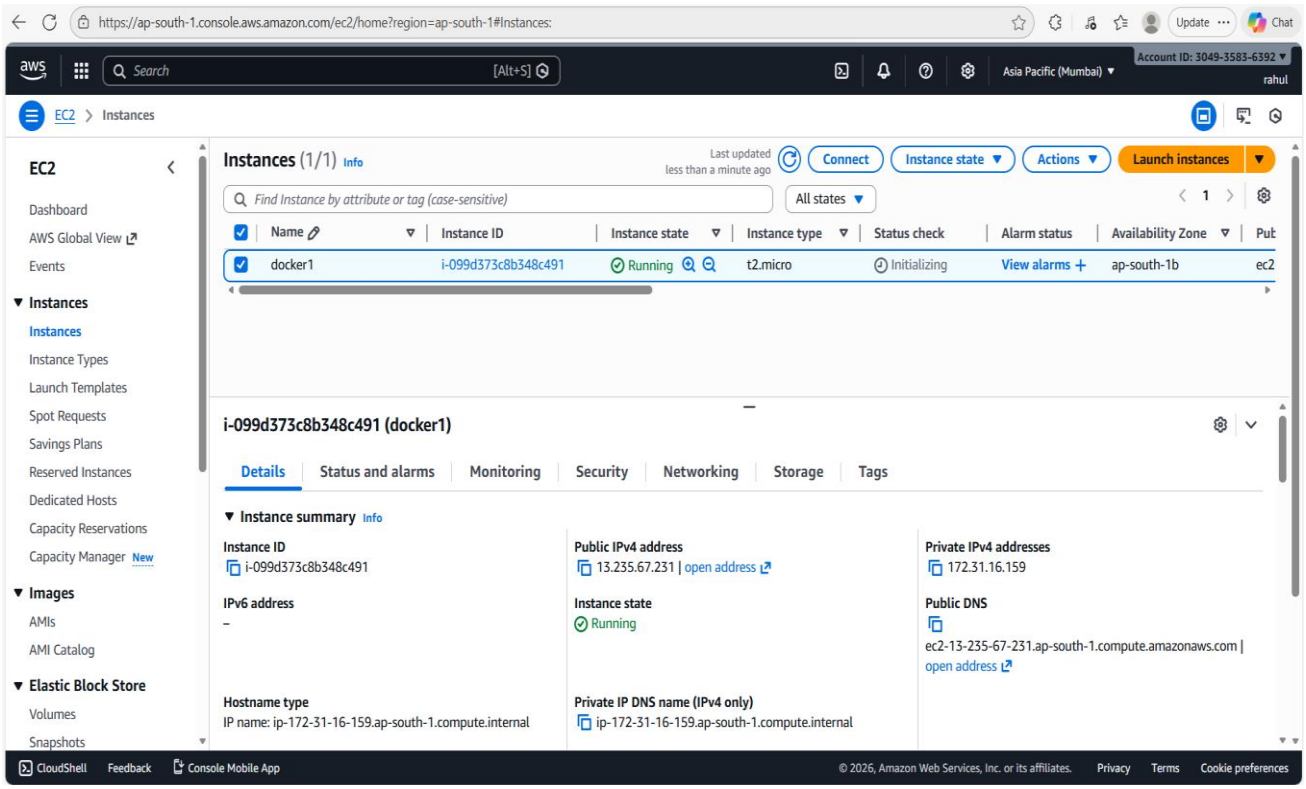
Solution

Step 1: Launch an EC2 Instance



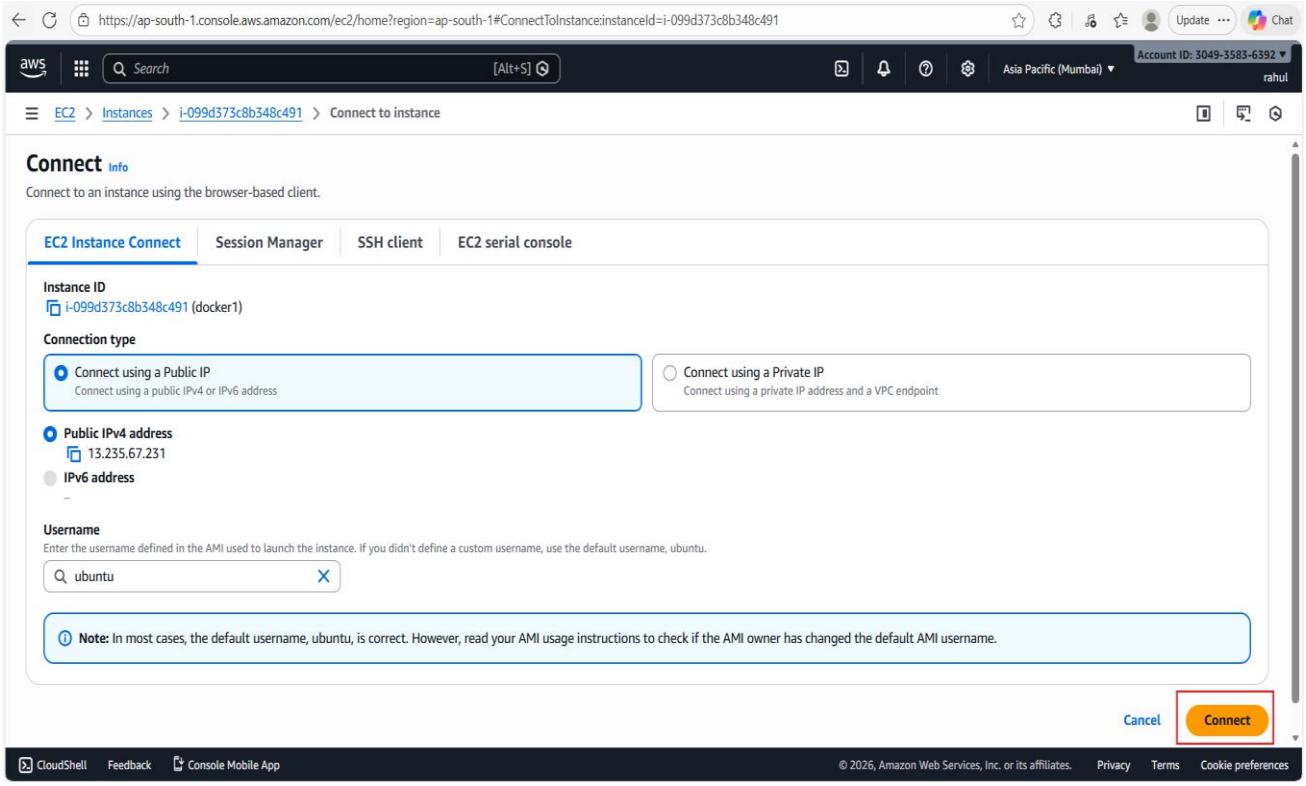
> **Comment:** *An AWS EC2 instance named "docker1" is being launched using the Ubuntu Server 22.04 LTS AMI (ami-087d1c9a513324697) on a t2.micro instance type in the Asia Pacific (Mumbai) region. The free-tier eligible configuration includes 1 volume of 8 GiB storage.*

Step 2: EC2 Instance Running



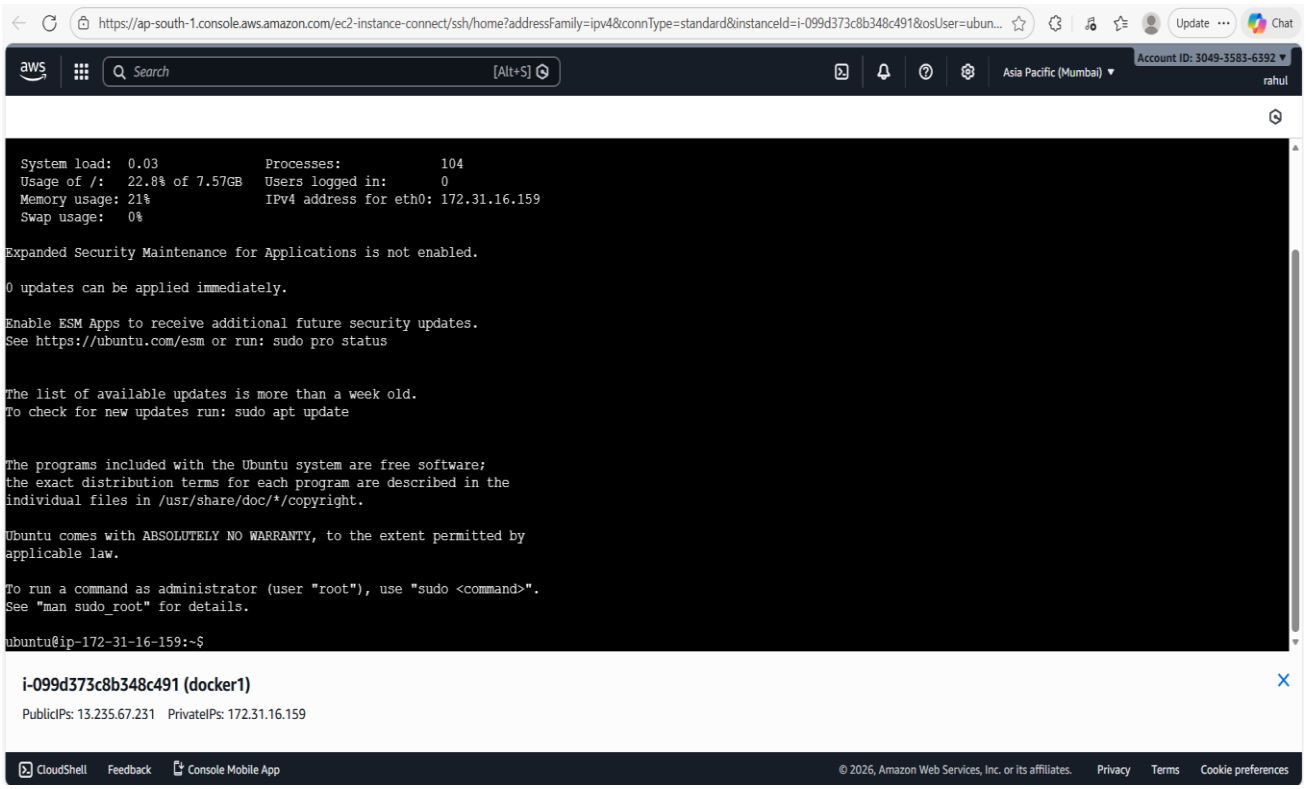
Comment: *The "docker1" EC2 instance (ID: i-099d373c8b348c491) is shown as "Running" in the AWS console. The public IPv4 address is 13.235.67.231 and the private IPv4 is 172.31.16.159, located in availability zone ap-south-1b.*

Step 3: Connect to the Instance



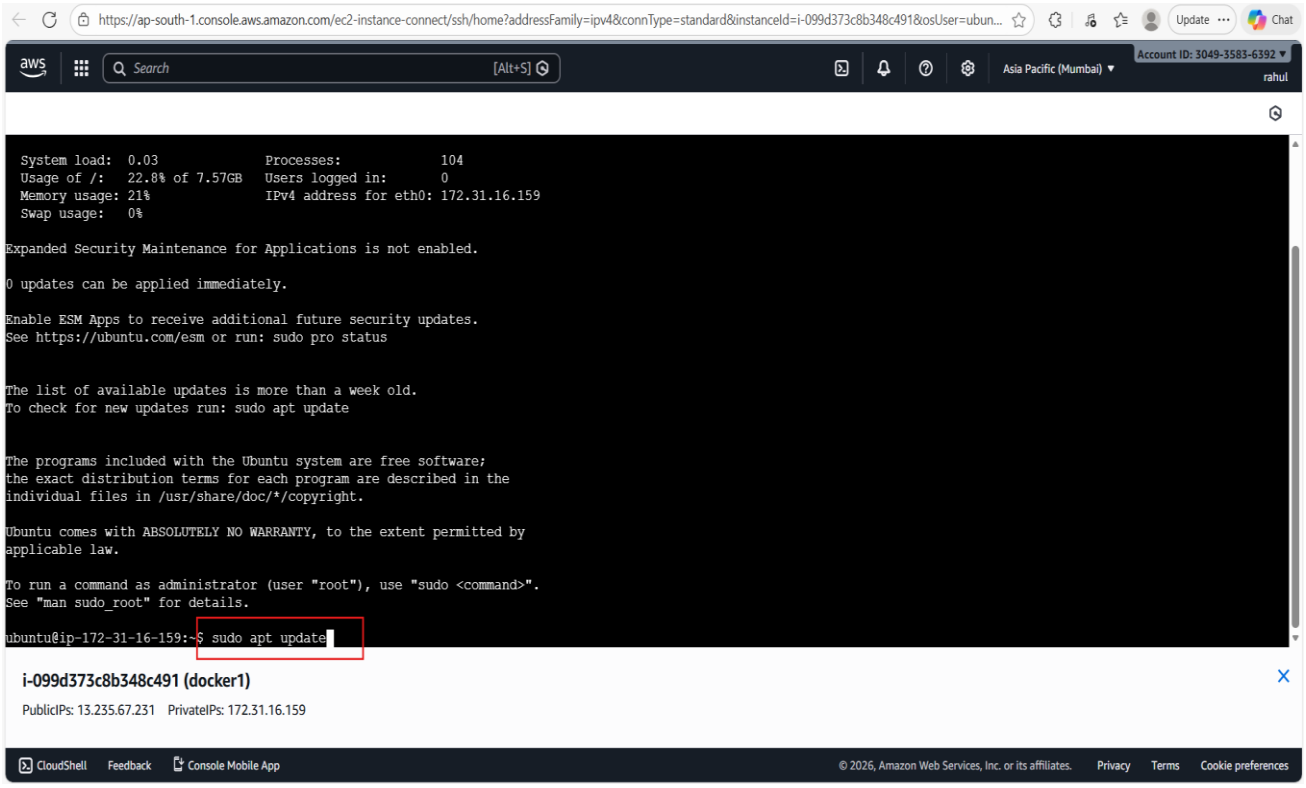
Comment: *Using EC2 Instance Connect to establish a browser-based SSH session. The instance is accessed using the default "ubuntu" username via the public IP 13.235.67.231.*

Step 4: SSH Session Opened



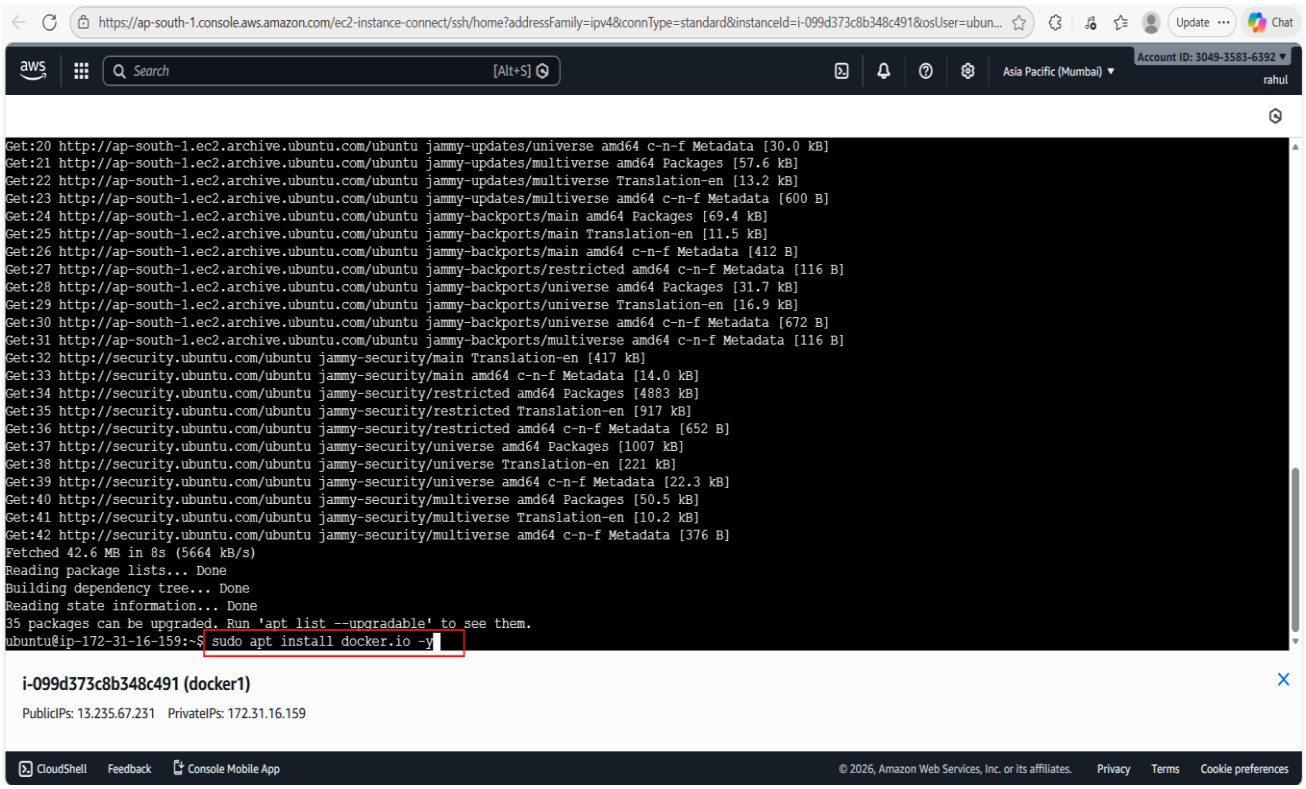
> **Comment:** *A successful SSH terminal session is opened inside the EC2 instance "docker1". The system shows minimal resource usage — 21% memory, 0% swap, 22.8% disk usage on a 7.57 GB volume.*

Step 5: Run sudo apt update



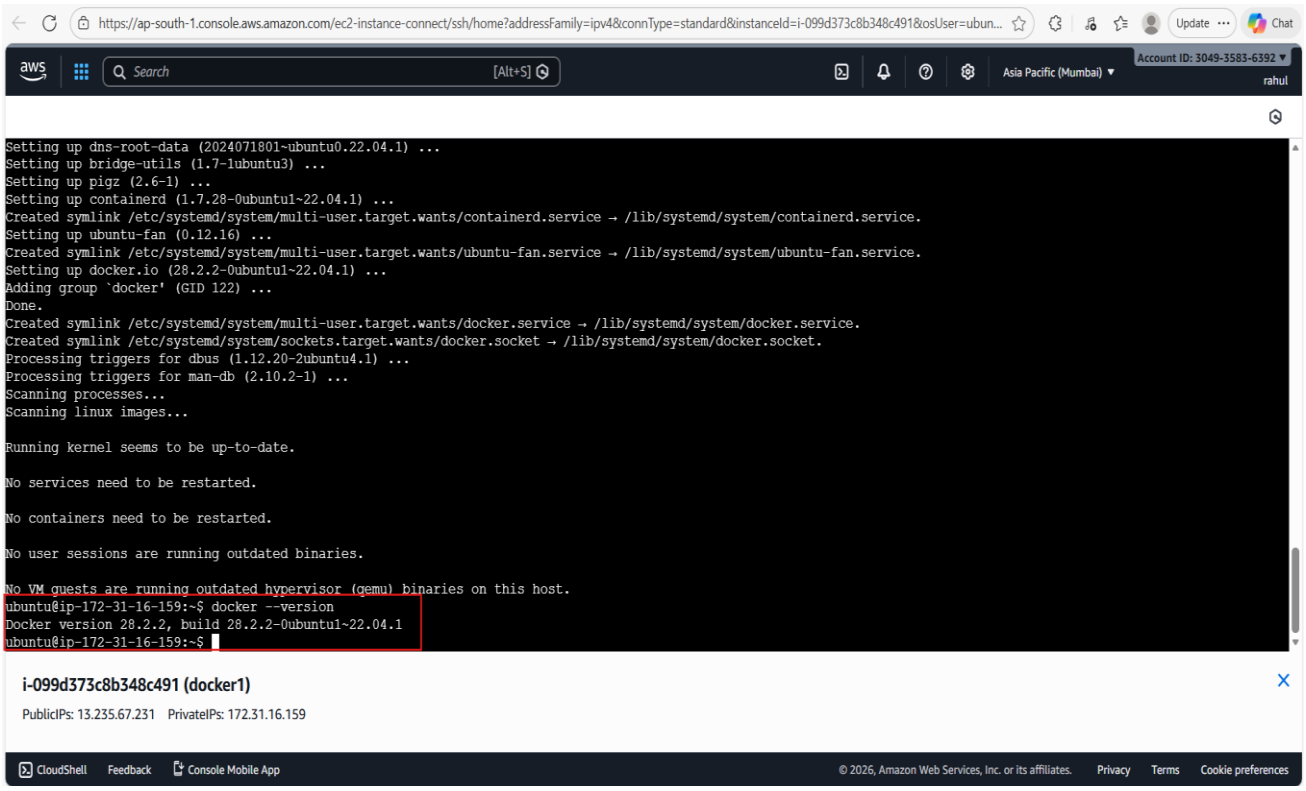
> **Comment:** *Running "sudo apt update" to refresh the package lists from Ubuntu repositories before installing Docker. This is a prerequisite step to ensure the latest package metadata is available.*

Step 6: Downloading Package Lists



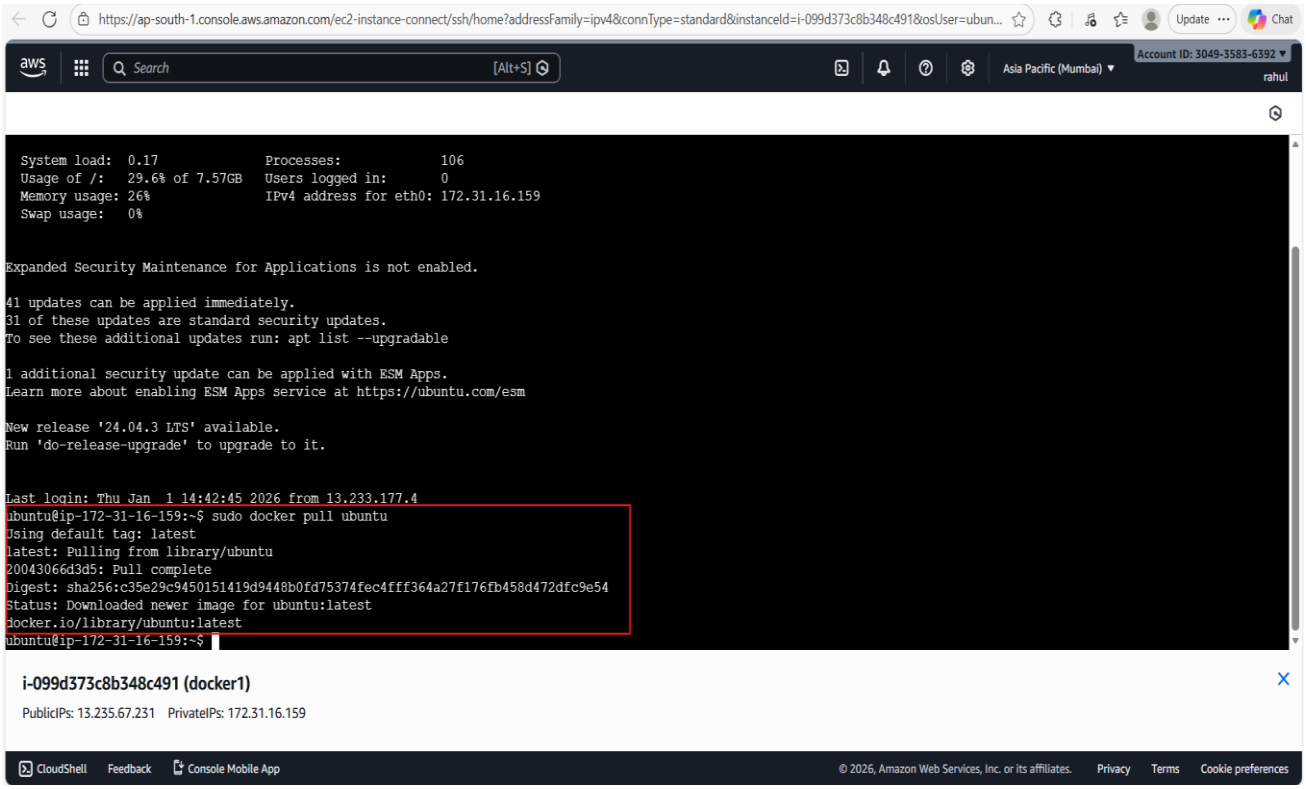
> **Comment:** *The apt update command fetches package metadata from Ubuntu's official archive and security repositories. A total of 42.6 MB was downloaded at 5664 kB/s. The next command "sudo apt install docker.io -y" is being executed to install Docker.*

Step 7: Docker Installed – Verify Version



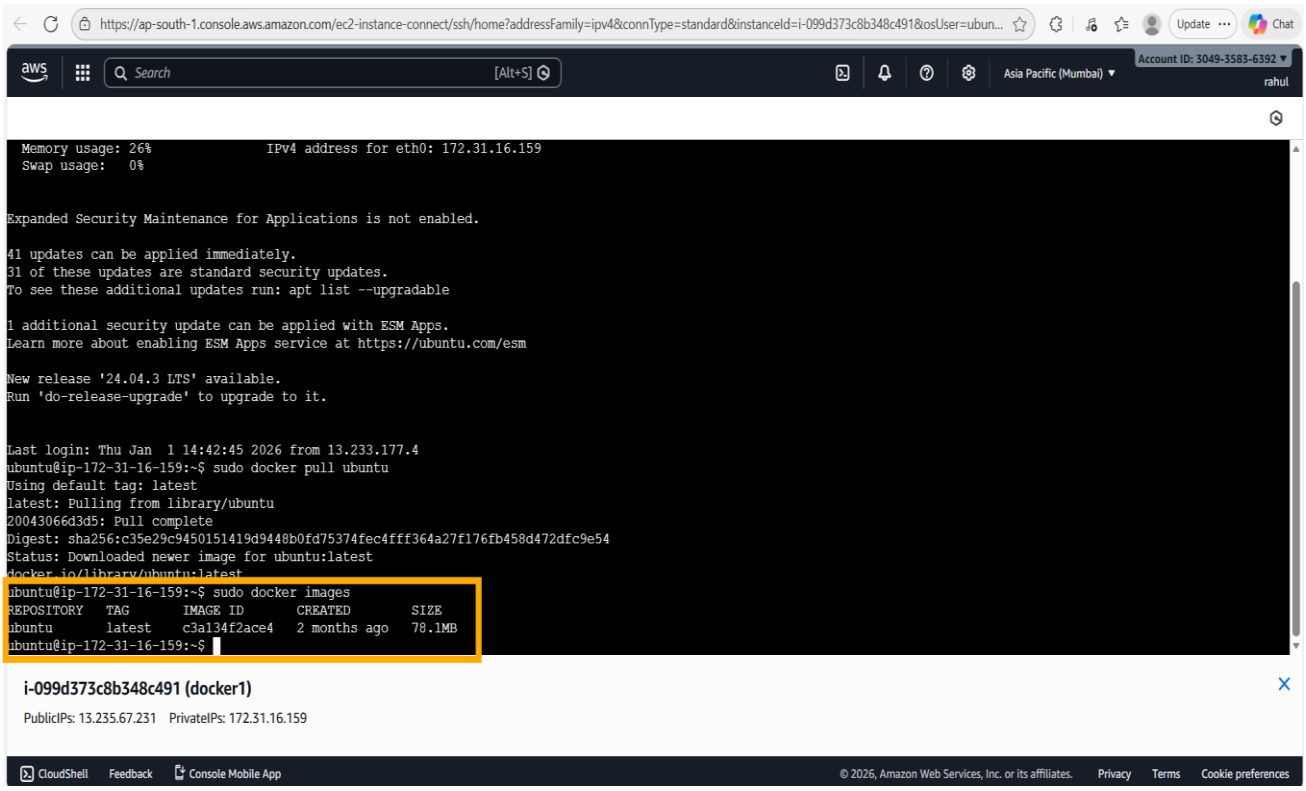
> **Comment:** *Docker (docker.io version 28.2.2) has been successfully installed on the Ubuntu EC2 instance. The output confirms Docker engine, containerd, and supporting services are all configured and active. Running "docker --version" confirms: Docker version 28.2.2, build 28.2.2-0ubuntu1~22.04.1.*

Step 8: Pull Ubuntu Docker Image



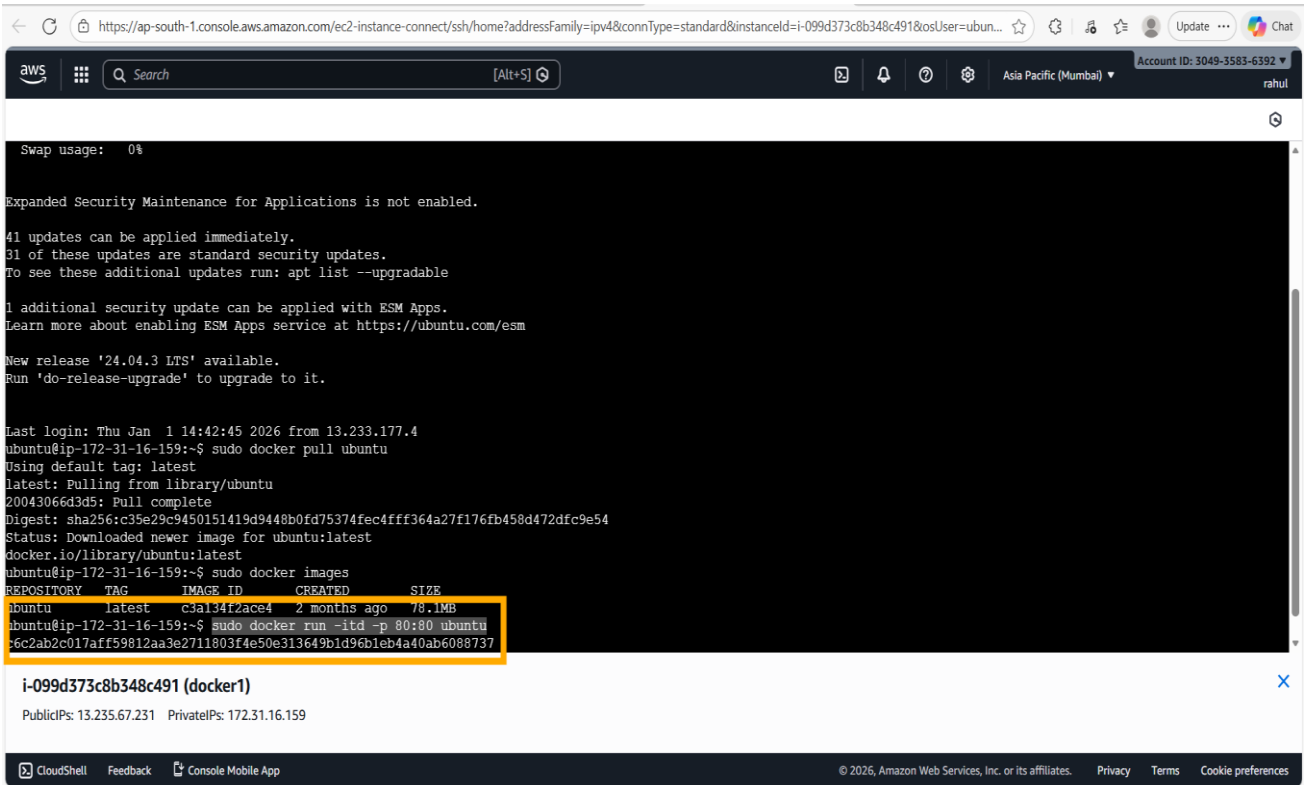
> **Comment:** *The command "sudo docker pull ubuntu" pulls the latest Ubuntu image from Docker Hub. The image (digest: sha256:c35e29c9...) is 78.1 MB and is downloaded successfully. Status confirms: "Downloaded newer image for ubuntu:latest".*

Step 9: Verify Docker Images



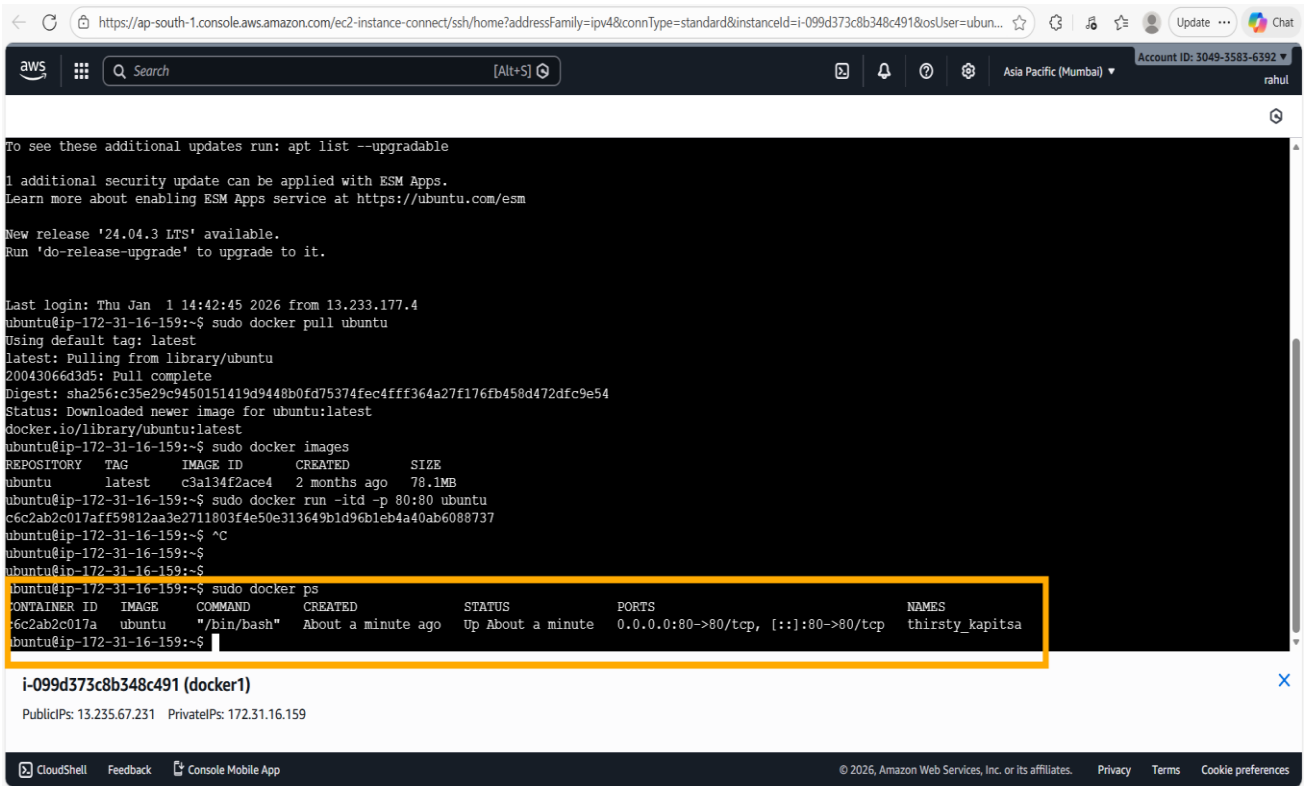
> **Comment:** *Running "sudo docker images" shows the ubuntu:latest image (ID: c3a134f2ace4, size: 78.1 MB) is now available locally. This confirms the image was successfully pulled from Docker Hub.*

Step 10: Run Ubuntu Container with Port 80 Mapped



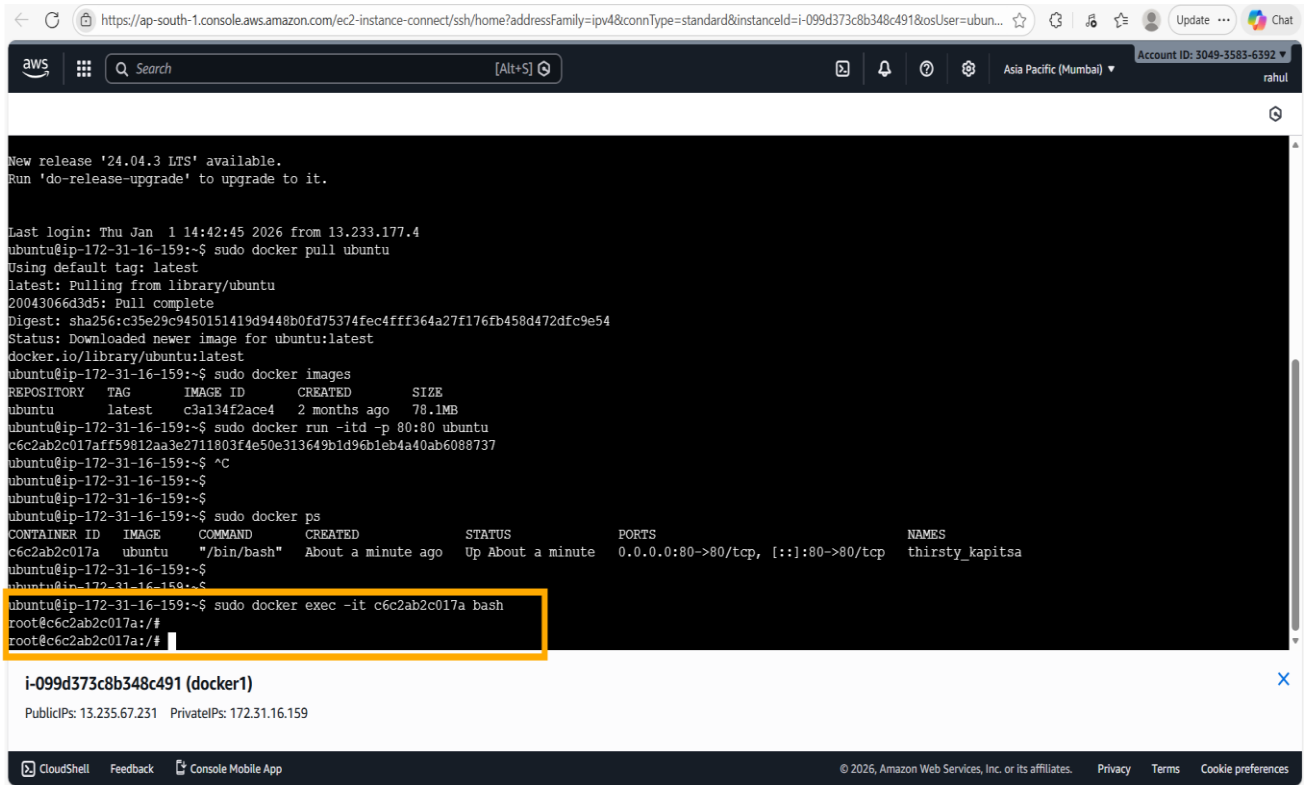
Comment: *The command "sudo docker run -itd -p 80:80 ubuntu" runs the Ubuntu container in detached interactive mode, mapping host port 80 to container port 80. The container ID c6c2ab2c017a is returned, confirming the container started successfully.*

Step 11: Verify Container is Running



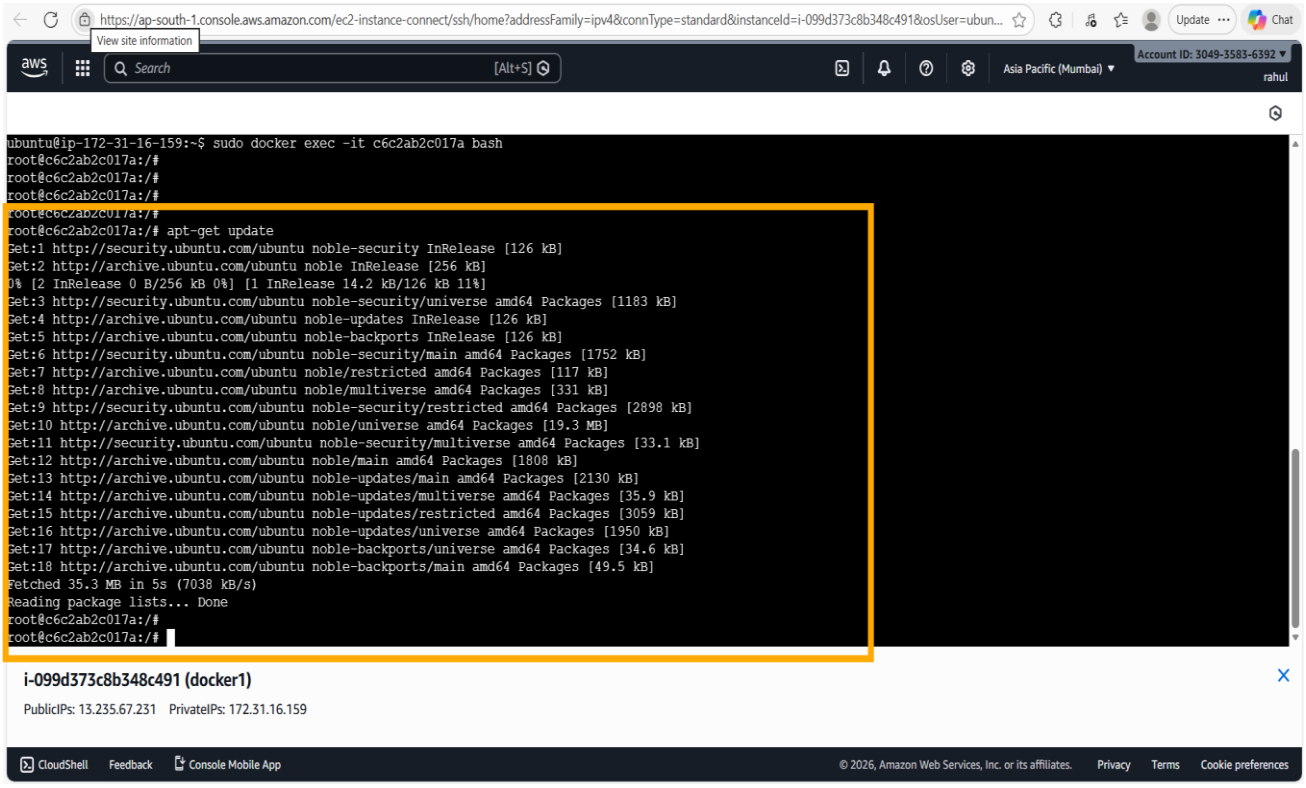
Comment: *Running "sudo docker ps" shows the container (c6c2ab2c017a) is active, using the Ubuntu image, mapped on ports 0.0.0.0:80->80/tcp and [::]:80->80/tcp. The container has been running for about 1 minute and is named "thirsty_kapitsa".*

Step 12: Exec into Container



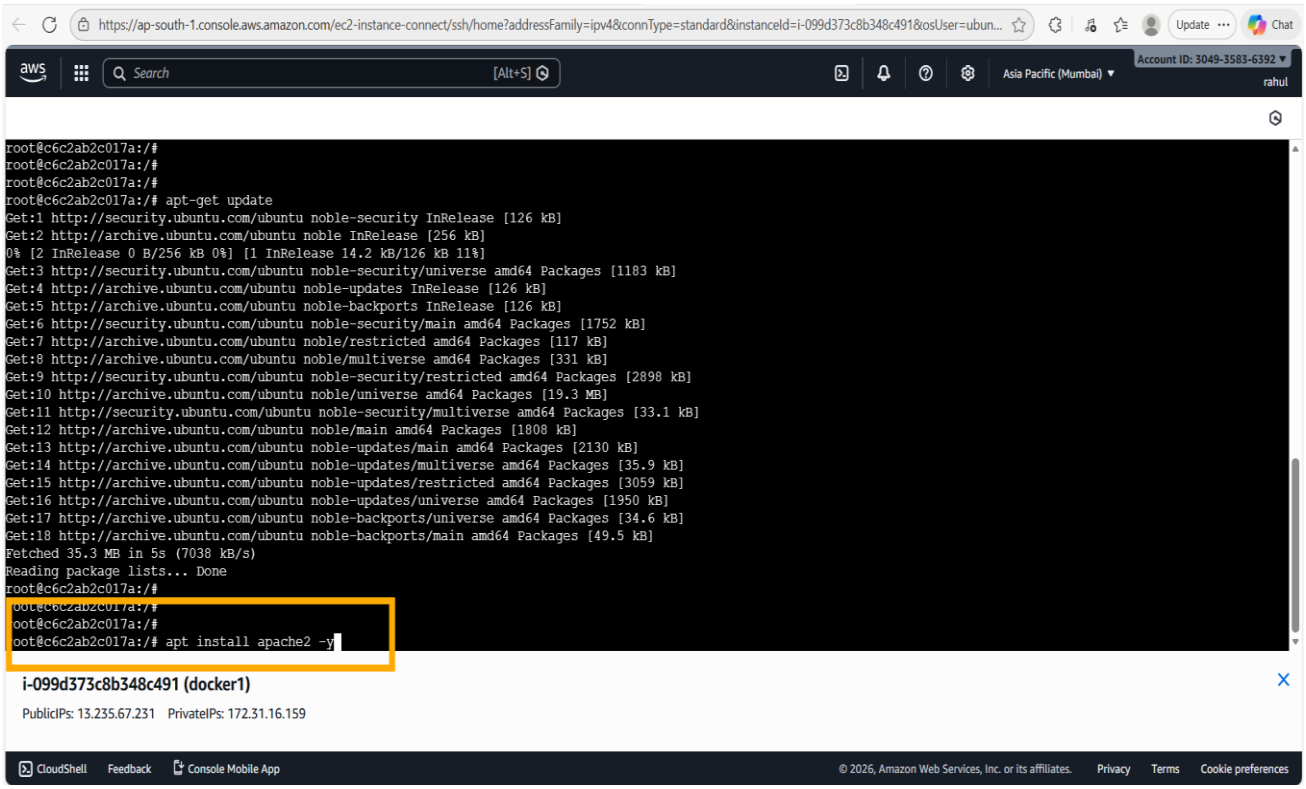
Comment: *The command "sudo docker exec -it c6c2ab2c017a bash" opens an interactive bash shell inside the running container. The prompt changes to "root@c6c2ab2c017a:/#", indicating we are now inside the container as root.*

Step 13: apt-get update Inside Container



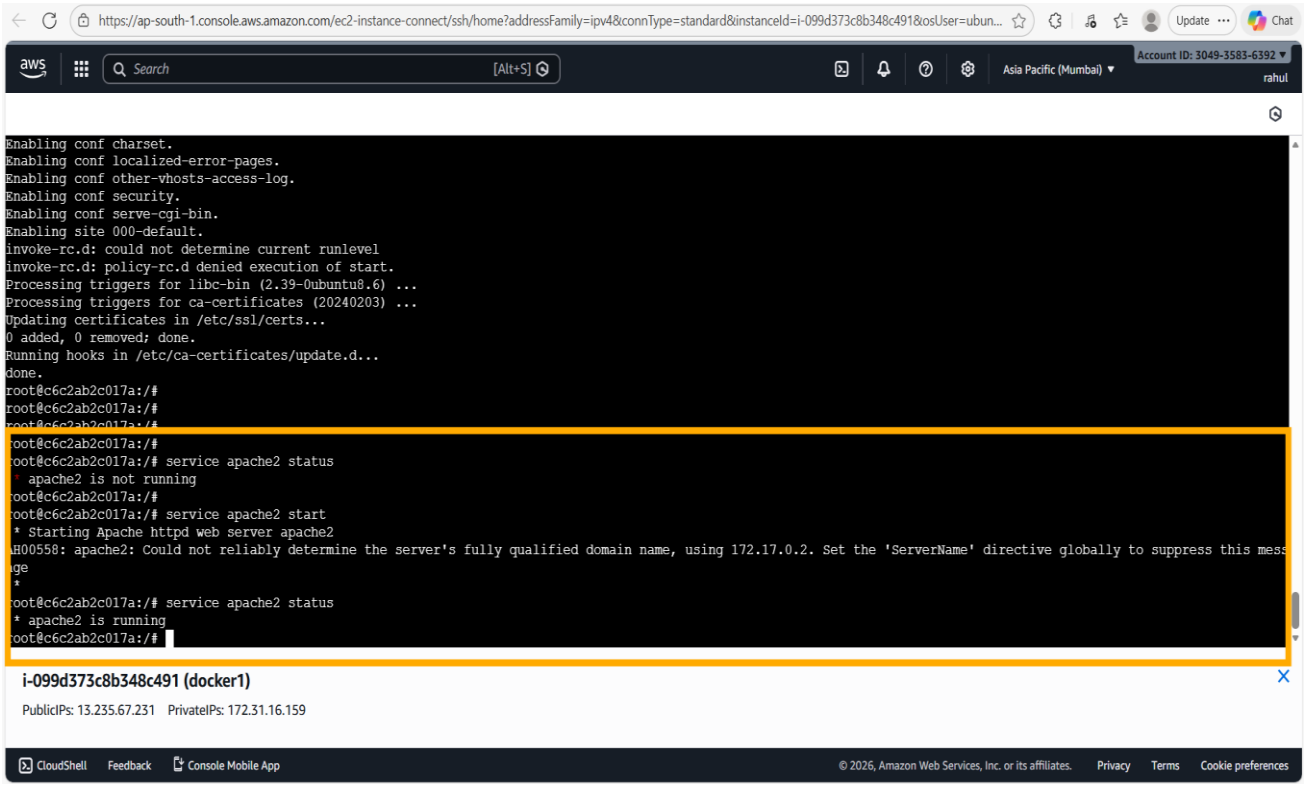
Comment: *Running "apt-get update" inside the Ubuntu container to refresh its package lists. Ubuntu Noble (24.04) repositories are fetched — 35.3 MB downloaded in 5 seconds at 7038 kB/s. This is required before installing Apache2.*

Step 14: Install Apache2 Inside Container



Comment: *The command "apt install apache2 -y" is executed inside the container to install the Apache2 web server with all its dependencies automatically accepted. This will install Apache2 and all required packages.*

Step 15: Start Apache2 Service



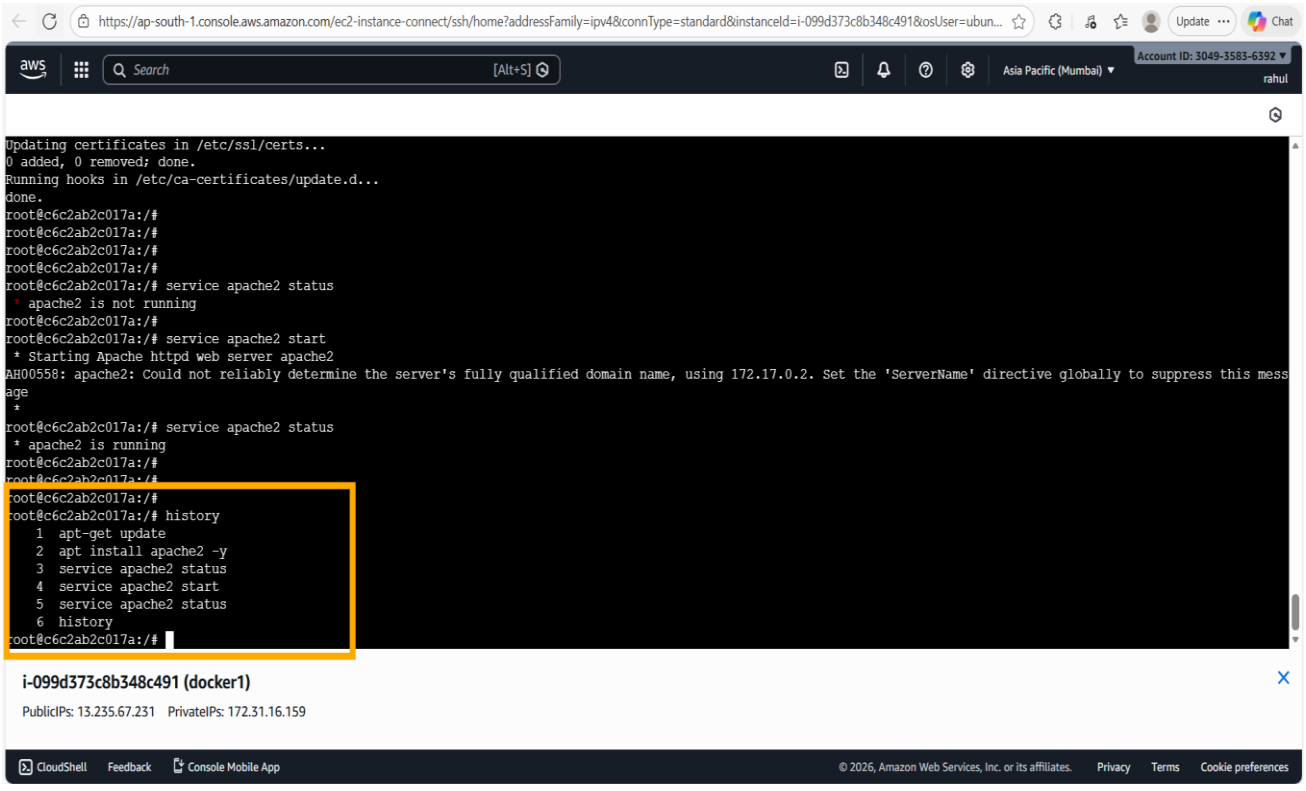
Comment: *Inside the container: "service apache2 status" shows apache2 is initially not running. After running "service apache2 start", the Apache HTTP web server starts successfully. A follow-up "service apache2 status" confirms "apache2 is running". Note: The AH00558 warning about ServerName is non-critical and does not affect functionality.*

Step 16: Verify Apache2 in Browser



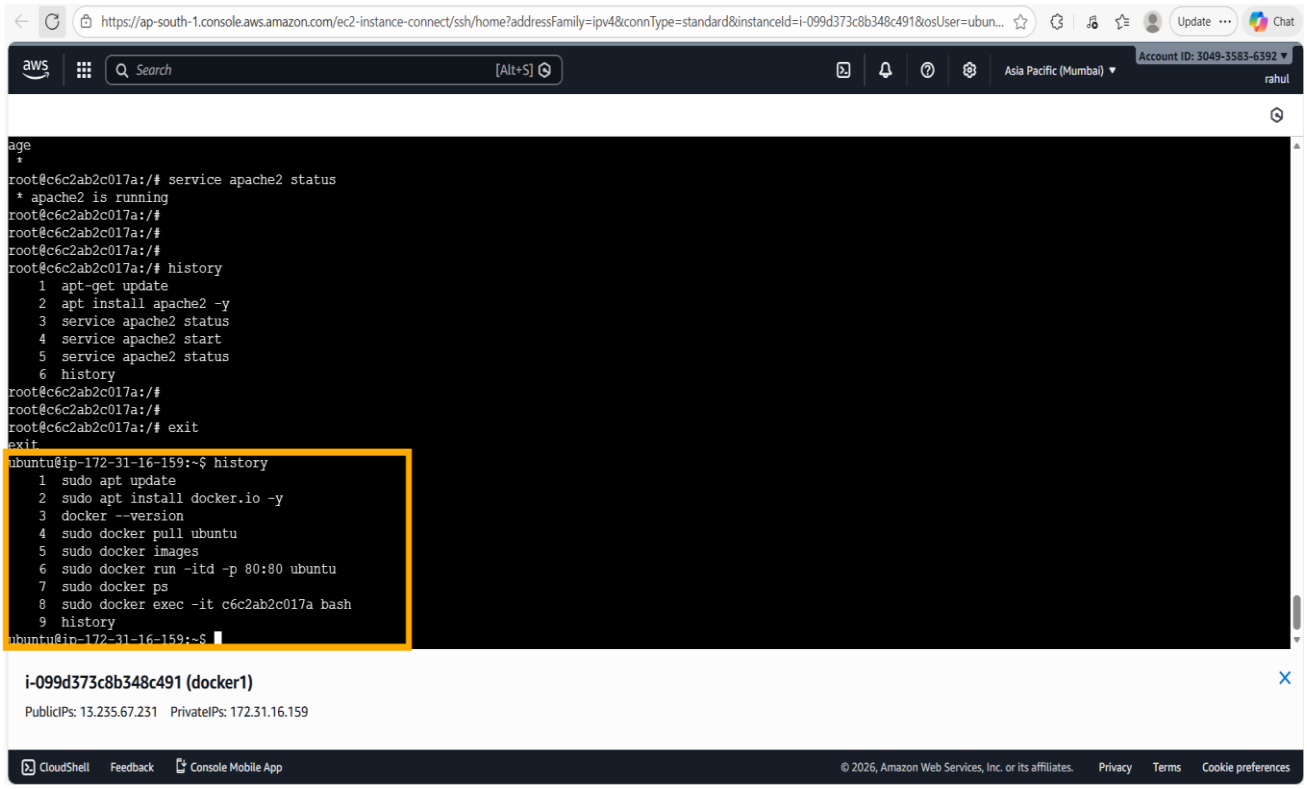
Comment: *The Apache2 default page ("It works!") is successfully accessible in the browser at the public IP 13.235.67.231 over port 80. This confirms that the Ubuntu container is running Apache2 and the port mapping is working correctly.*

Step 17: Command History Inside Container



Comment: *The "history" command inside the container shows the full sequence of commands run: apt-get update → apt install apache2 -y → service apache2 status → service apache2 start → service apache2 status. This provides a clear audit trail of all steps performed.*

Step 18: Command History on Host EC2 Instance



Comment: *The host EC2 instance command history shows the complete workflow: sudo apt update → sudo apt install docker.io -y → docker --version → sudo docker pull ubuntu → sudo docker images → sudo docker run -itd -p 80:80 ubuntu → sudo docker ps → sudo docker exec -it c6c2ab2c017a bash. This documents the full end-to-end process.*

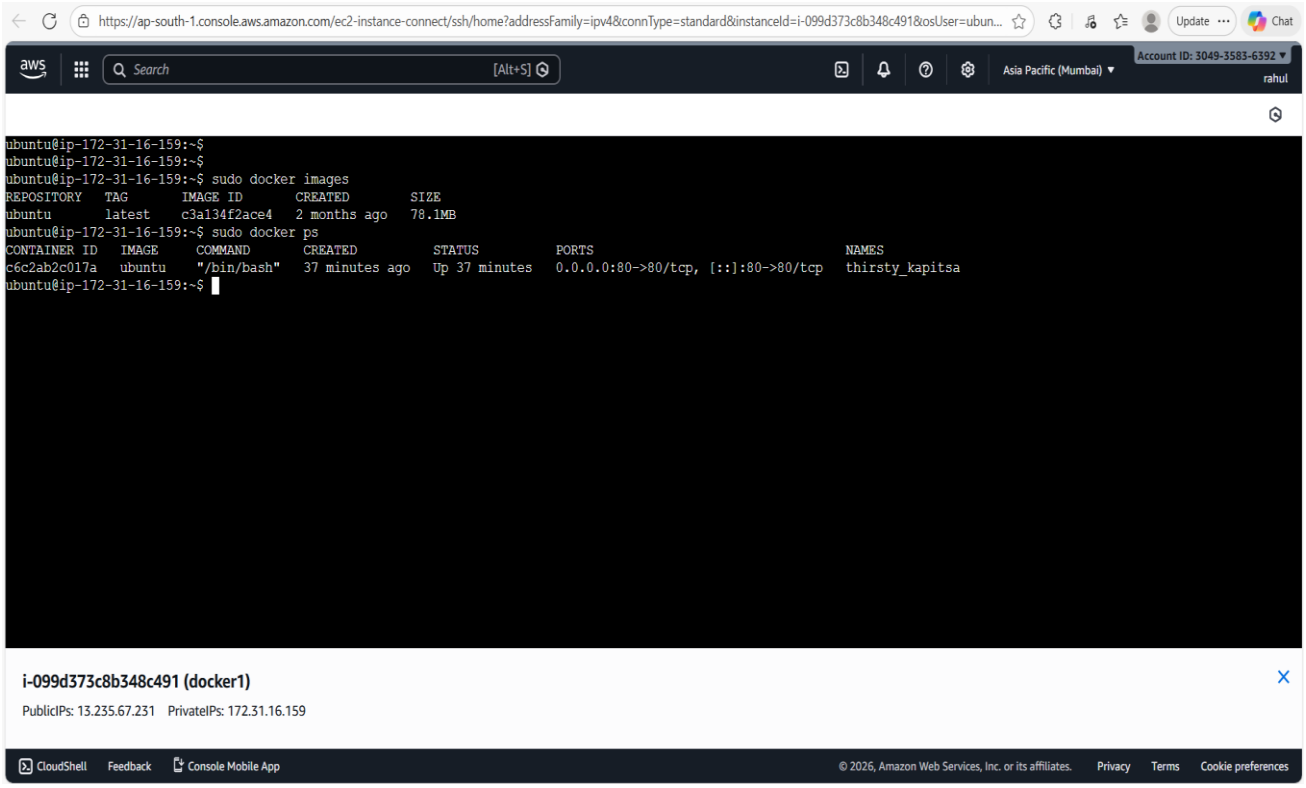
Assignment 2 – Save Docker Image & Launch on Port 81

Tasks To Be Performed:

1. Save the image created in Assignment 1 as a Docker image
2. Launch a container from this new image and map the port to 81
3. Go inside the container and start the Apache2 service
4. Check if you are able to access it on the browser

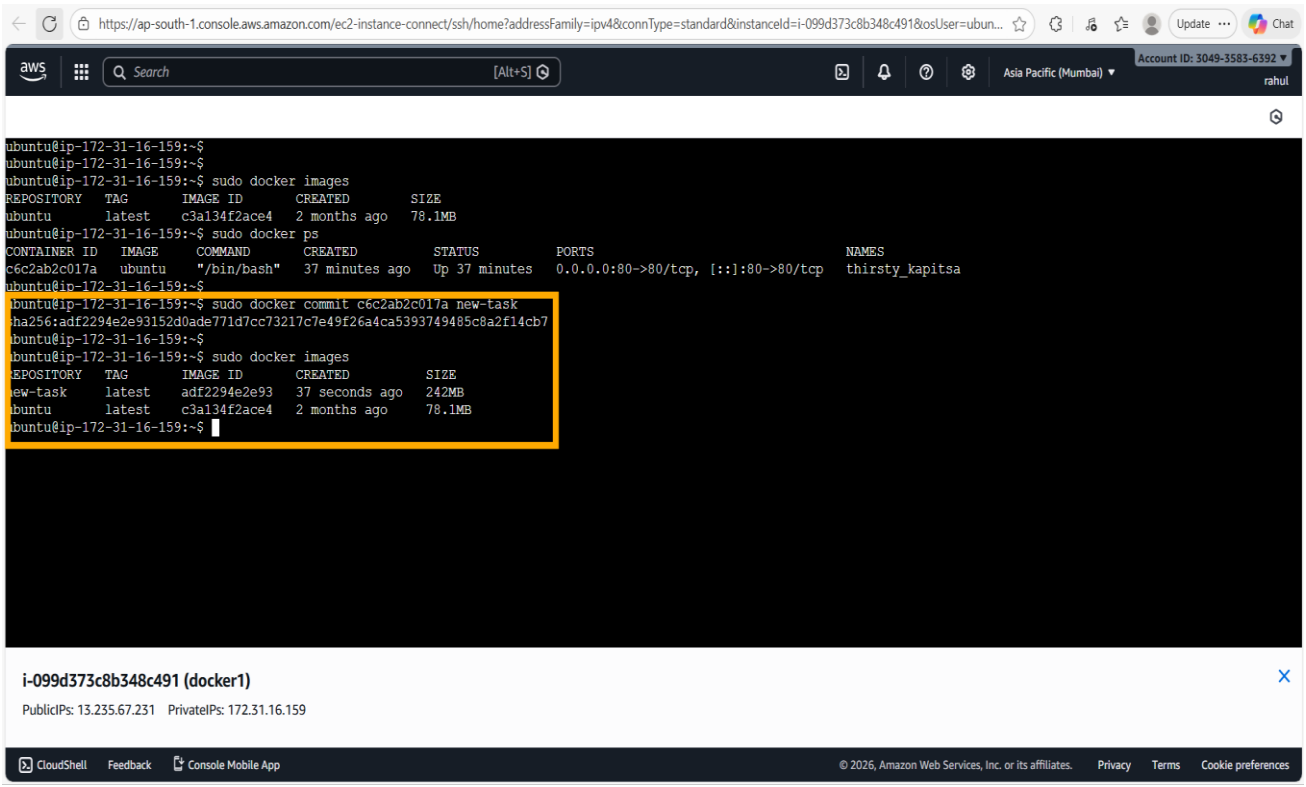
Solution

Step 1: Check Existing Images and Running Containers



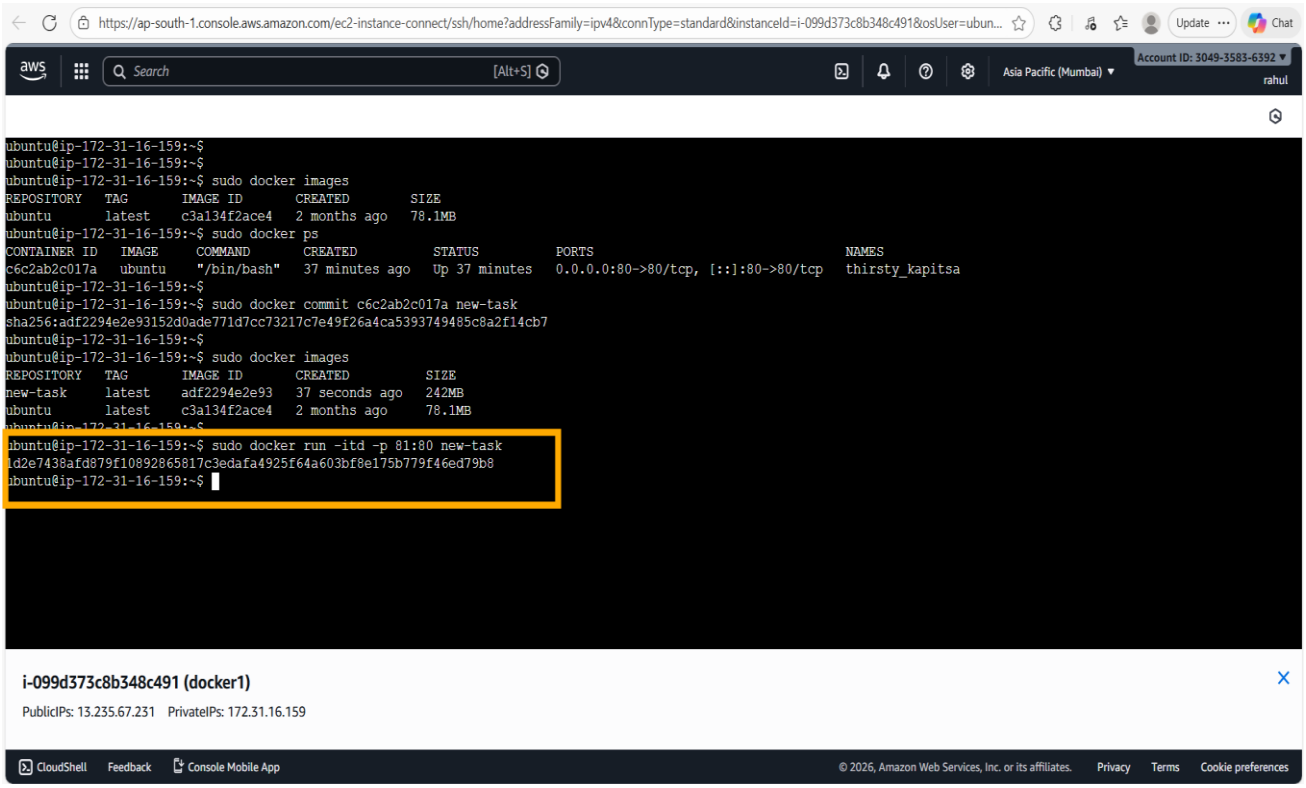
> **Comment:** *Running "sudo docker images" and "sudo docker ps" shows the ubuntu image (78.1 MB) and the running container c6c2ab2c017a (port 0.0.0.0:80->80/tcp) from Assignment 1. This container has Apache2 already installed and is the source for the image commit.*

Step 2: Commit the Container as a New Docker Image



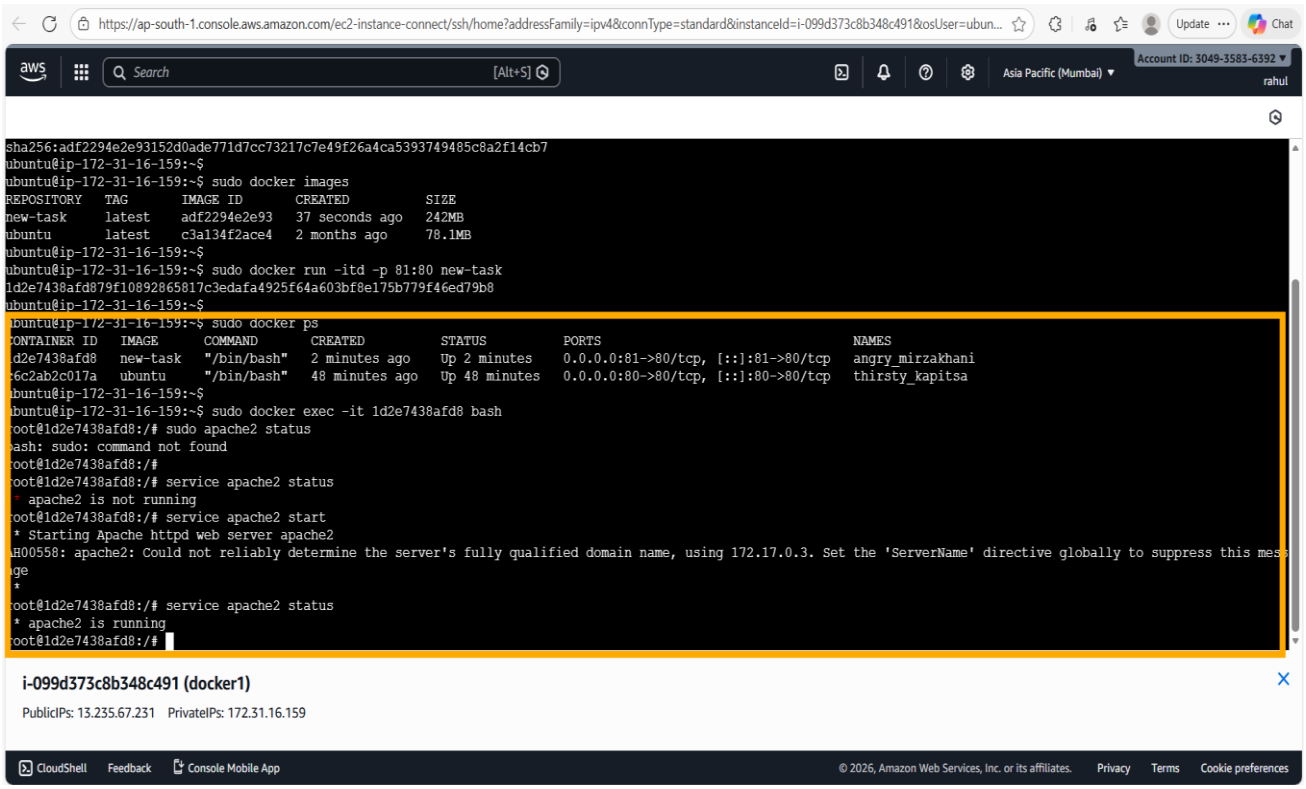
Comment: *The command "sudo docker commit c6c2ab2c017a new-task" creates a new Docker image from the running container's current state (with Apache2 installed). The new image "new-task" (ID: adf2294e2e93, size: 242 MB) is now listed alongside the original ubuntu image. The size increase from 78.1 MB to 242 MB reflects the Apache2 installation baked into the image.*

Step 3: Run New Container on Port 81



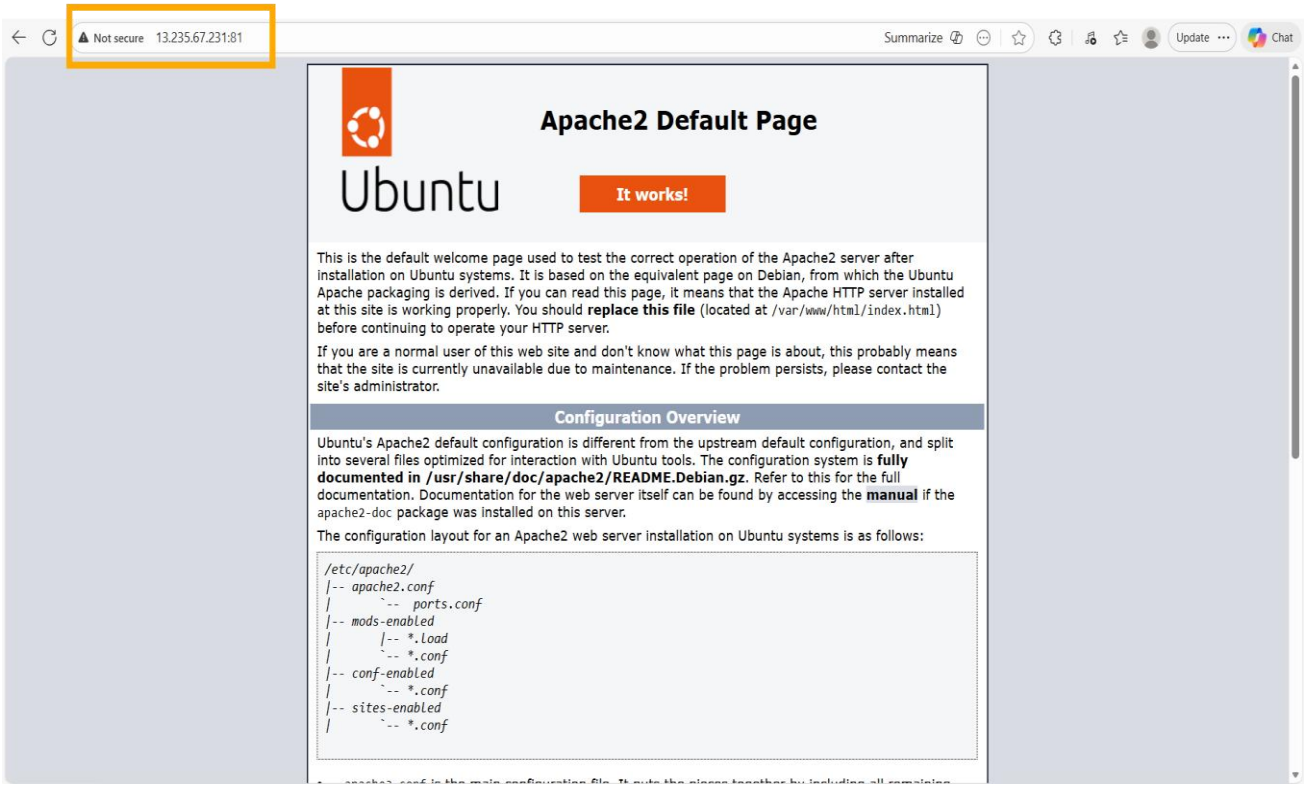
Comment: *The command "sudo docker run -itd -p 81:80 new-task" launches a new container from the saved "new-task" image, mapping host port 81 to container port 80. A new container ID (1d2e7438afd8) is generated, confirming the container started successfully.*

Step 4: Exec into New Container & Start Apache2



Comment: *"sudo docker ps" shows both containers running: the new container (1d2e7438afd8) using "new-task" image on port 81:80, and the original (c6c2ab2c017a) on port 80:80. After exec-ing into the new container, "service apache2 status" shows it is not running initially (since it is a fresh instance). Running "service apache2 start" starts Apache2, and the follow-up status check confirms "apache2 is running".*

Step 5: Verify Apache2 on Port 81 in Browser



Comment: *The Apache2 default page is successfully accessible at 13.235.67.231:81 in the browser, confirming that the new container (launched from the saved "new-task" image) is serving on port 81 as expected. This validates the Docker image commit and port remapping workflow.*

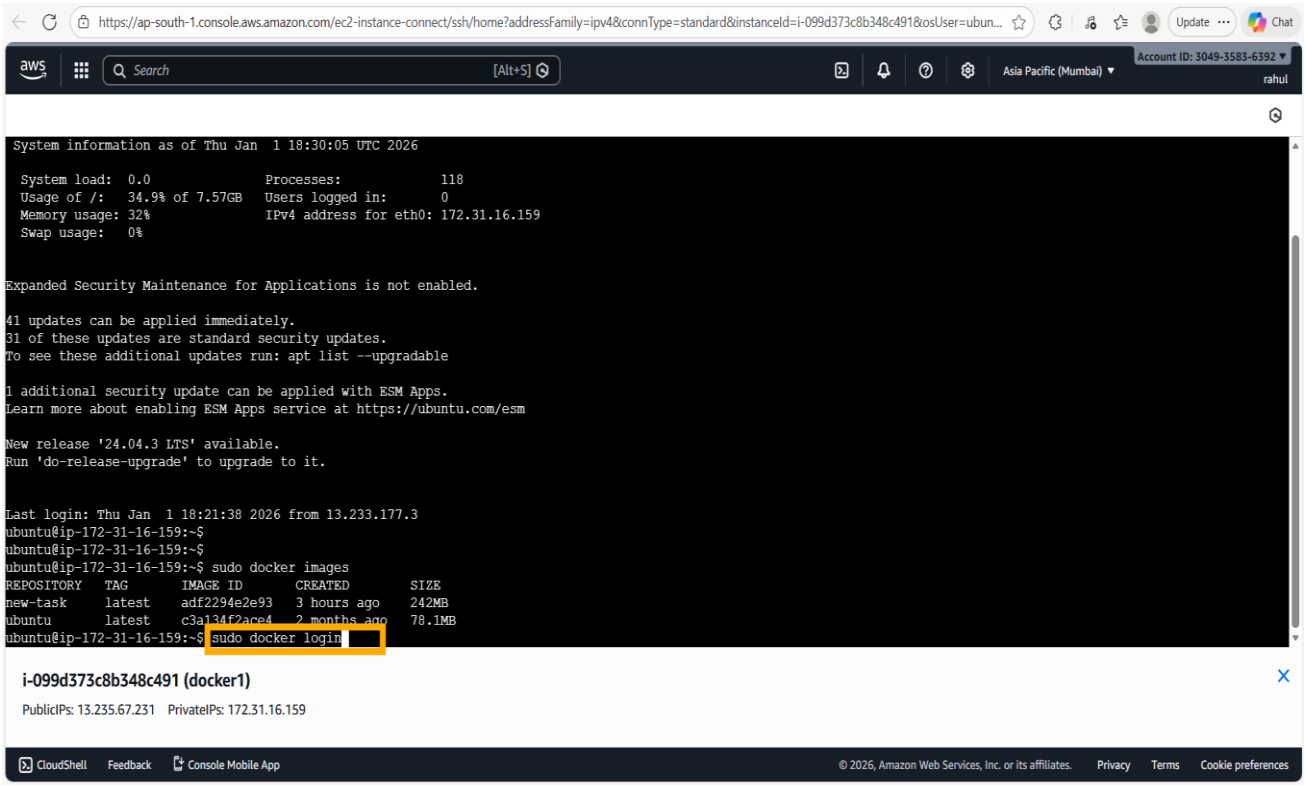
Assignment 3 – Push Image to Docker Hub & Pull on New Machine

Tasks To Be Performed:

1. Use the saved image from the previous assignment
2. Upload this image to Docker Hub
3. On a separate machine, pull this Docker Hub image and launch it on port 80
4. Start the Apache2 service
5. Verify if you are able to see the Apache2 service

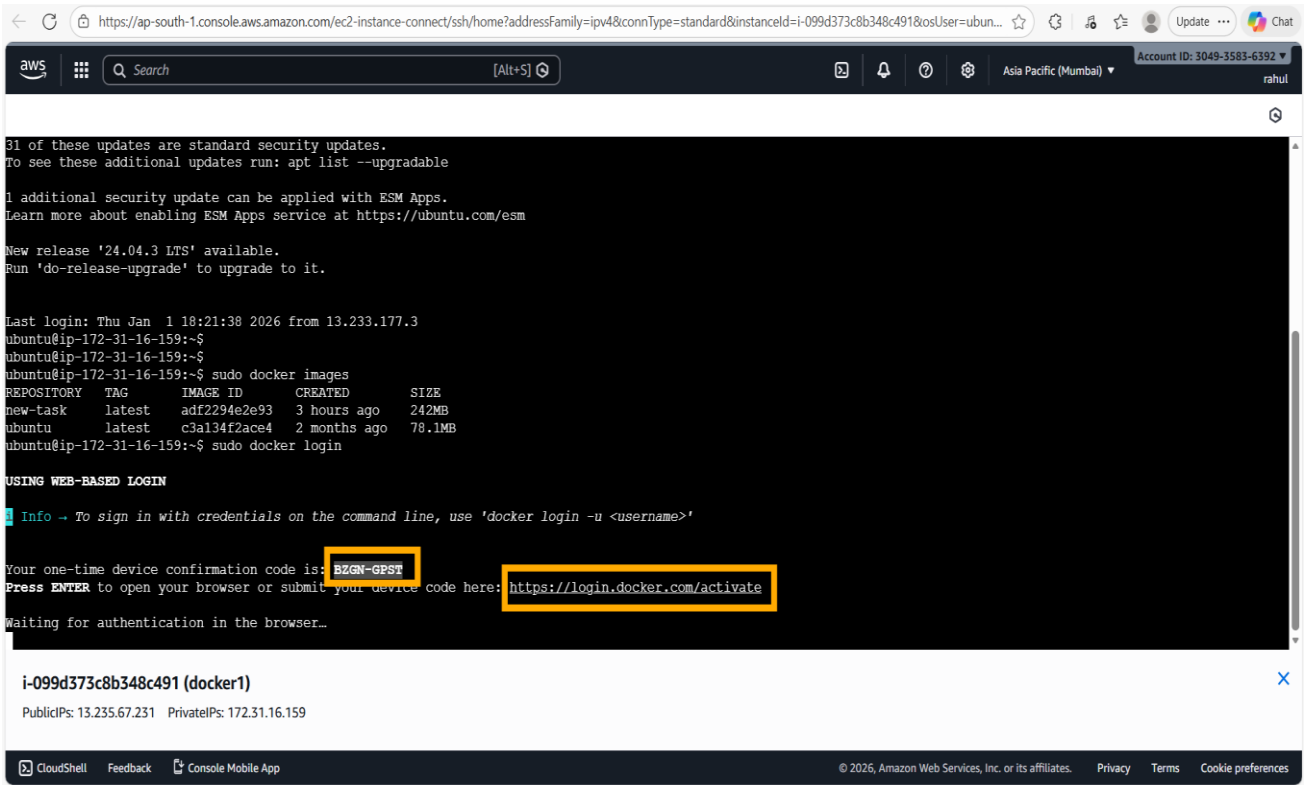
Solution

Step 1: Check Images & Initiate Docker Login



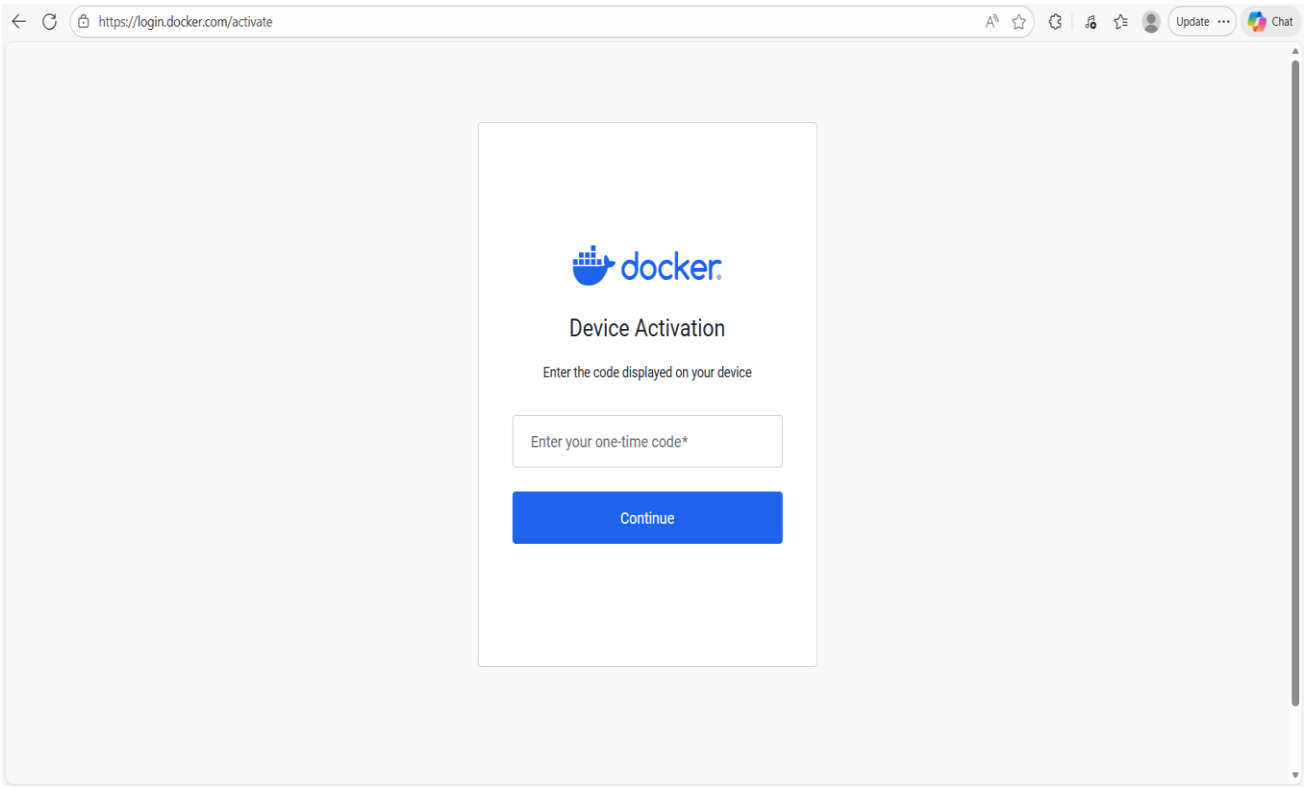
> **Comment:** *"sudo docker images" confirms the "new-task" image (242 MB) and "ubuntu" (78.1 MB) are available on the host. The "sudo docker login" command is initiated to authenticate with Docker Hub before pushing the image.*

Step 2: Docker Hub Web-Based Login – One-Time Code



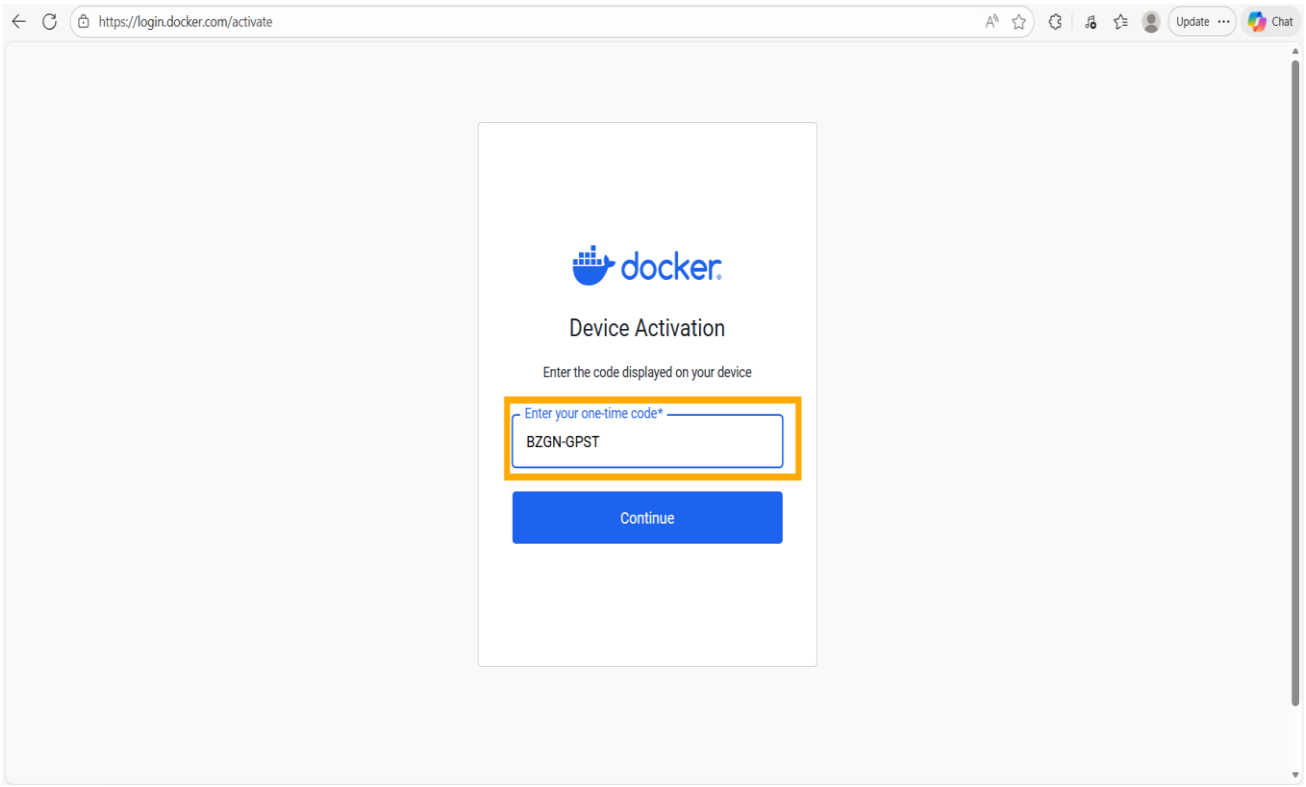
> **Comment:** *Docker uses web-based login flow. A one-time device confirmation code "BZGN-GPST" is displayed. The user is directed to https://login.docker.com/activate to complete authentication. The terminal waits for browser-based confirmation.*

Step 3: Docker Hub Device Activation Page



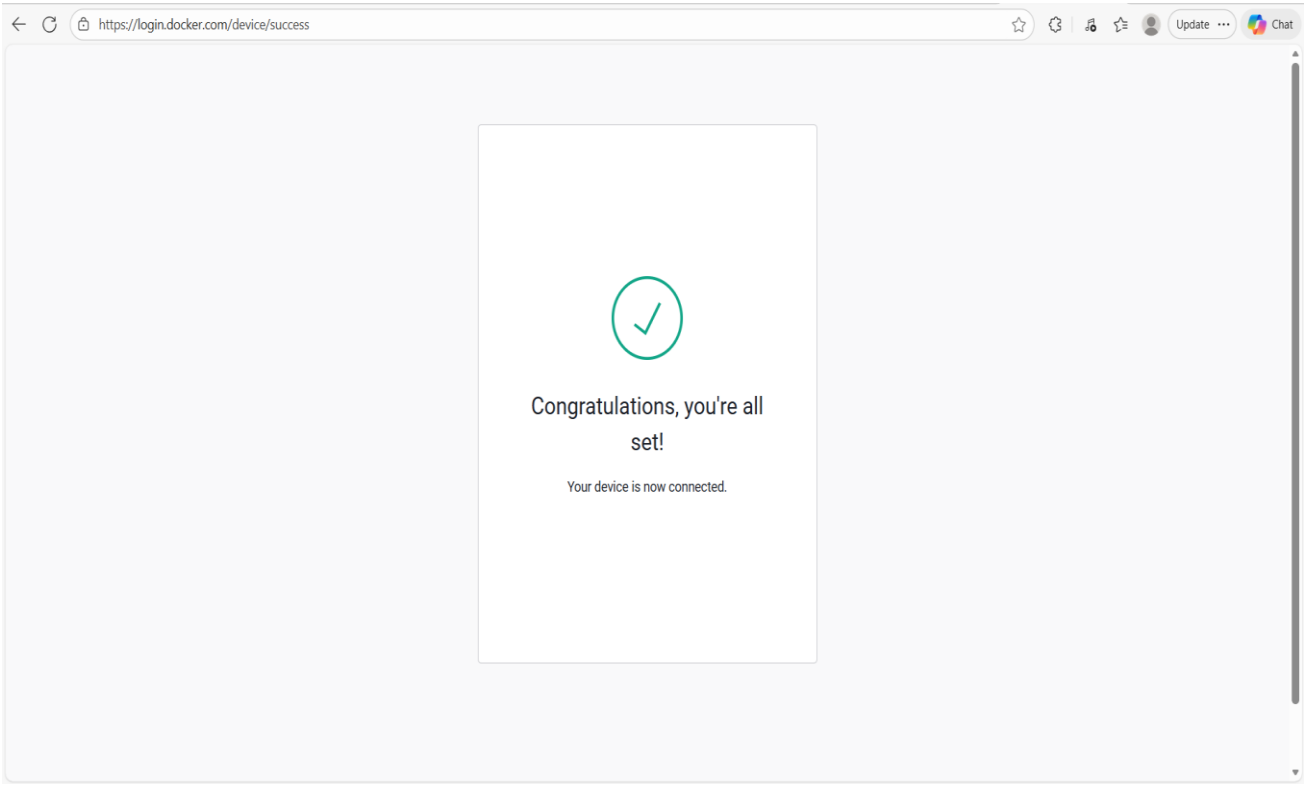
> **Comment:** *The Docker Hub Device Activation page at https://login.docker.com/activate is opened in the browser, prompting the user to enter the one-time code displayed in the terminal.*

Step 4: Enter One-Time Code



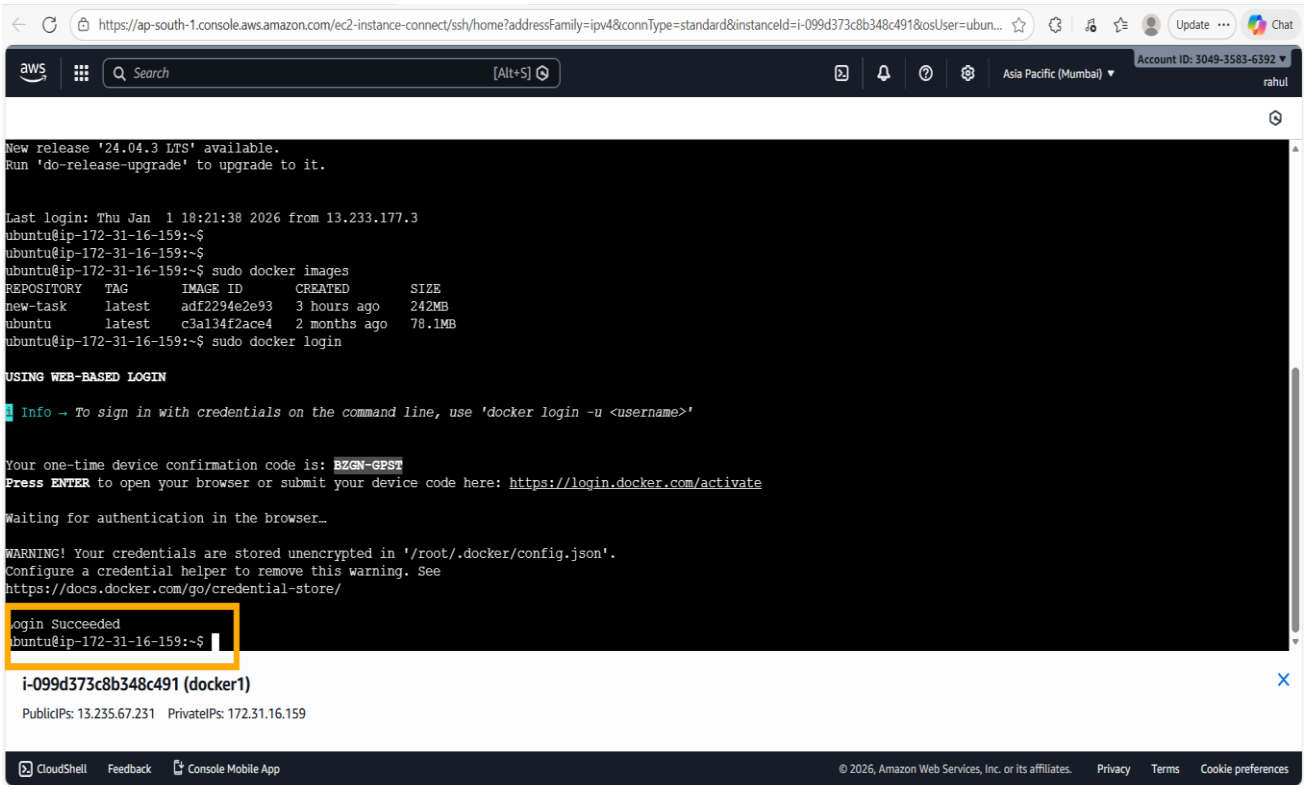
> **Comment:** *The one-time code "BZGN-GPST" is entered into the Docker Device Activation form. Clicking "Continue" will authorize the CLI session with the Docker Hub account.*

Step 5: Device Successfully Connected



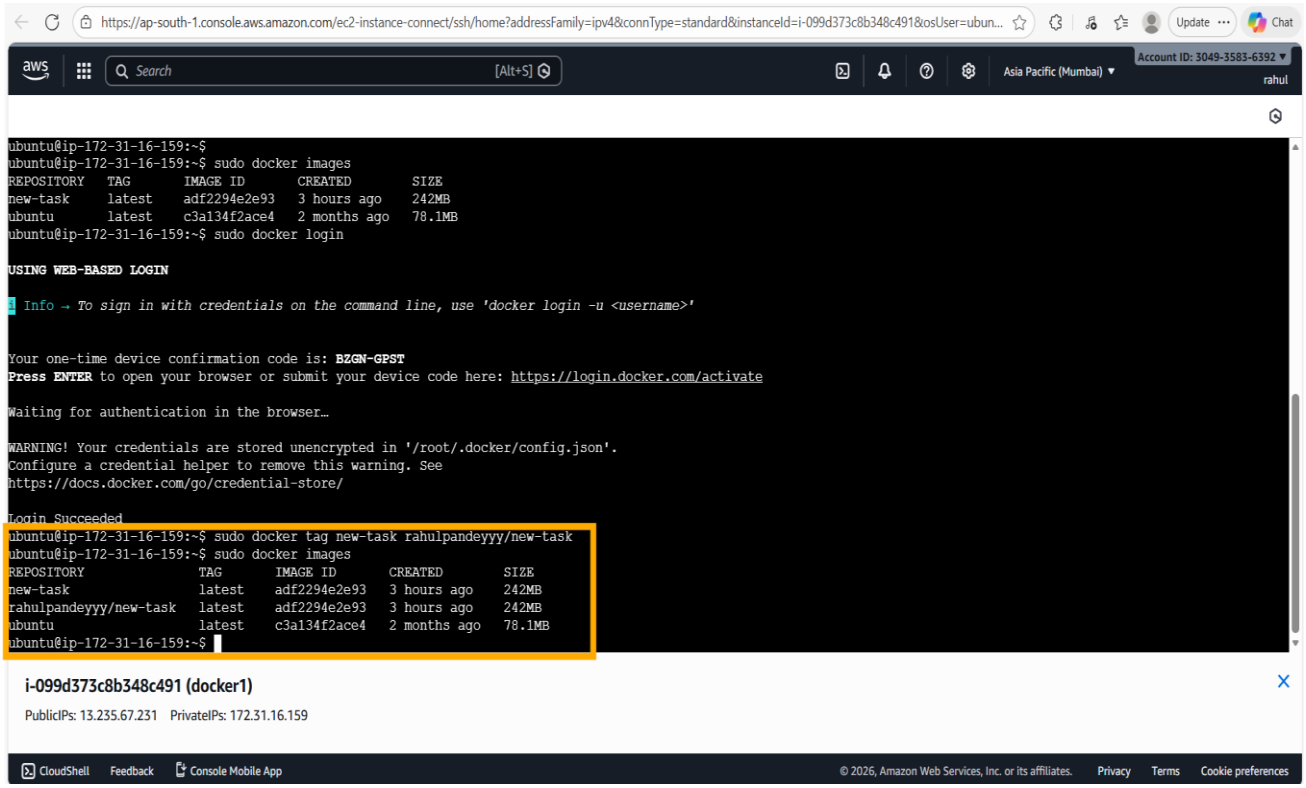
> **Comment:** *The Docker Hub device activation is successful. The page at https://login.docker.com/device/success shows "Congratulations, you're all set! Your device is now connected." The terminal CLI is now authenticated with Docker Hub.*

Step 6: Login Succeeded – Tag the Image



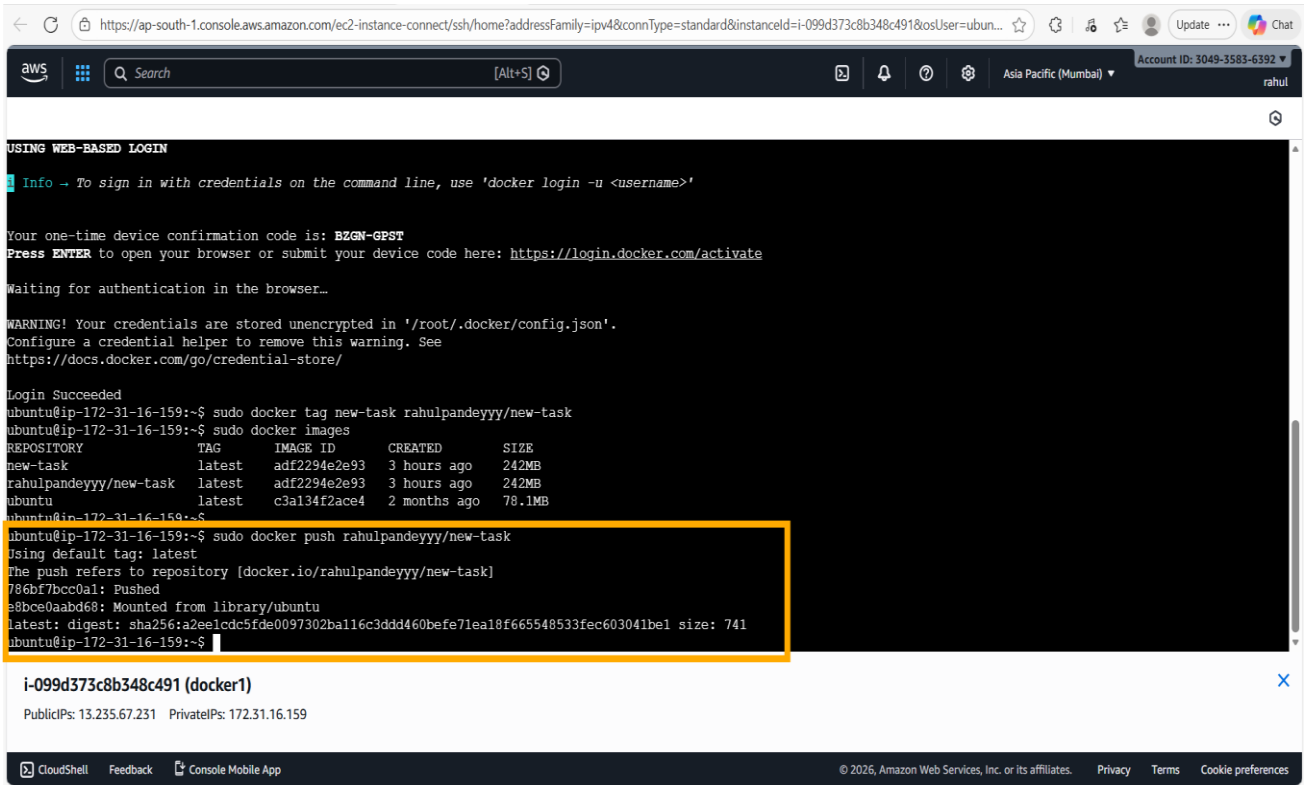
> **Comment:** *"Login Succeeded" is shown in the terminal, confirming the Docker Hub authentication is complete. Note: Credentials are stored in /root/.docker/config.json.*

Step 7: Tag Image and Verify



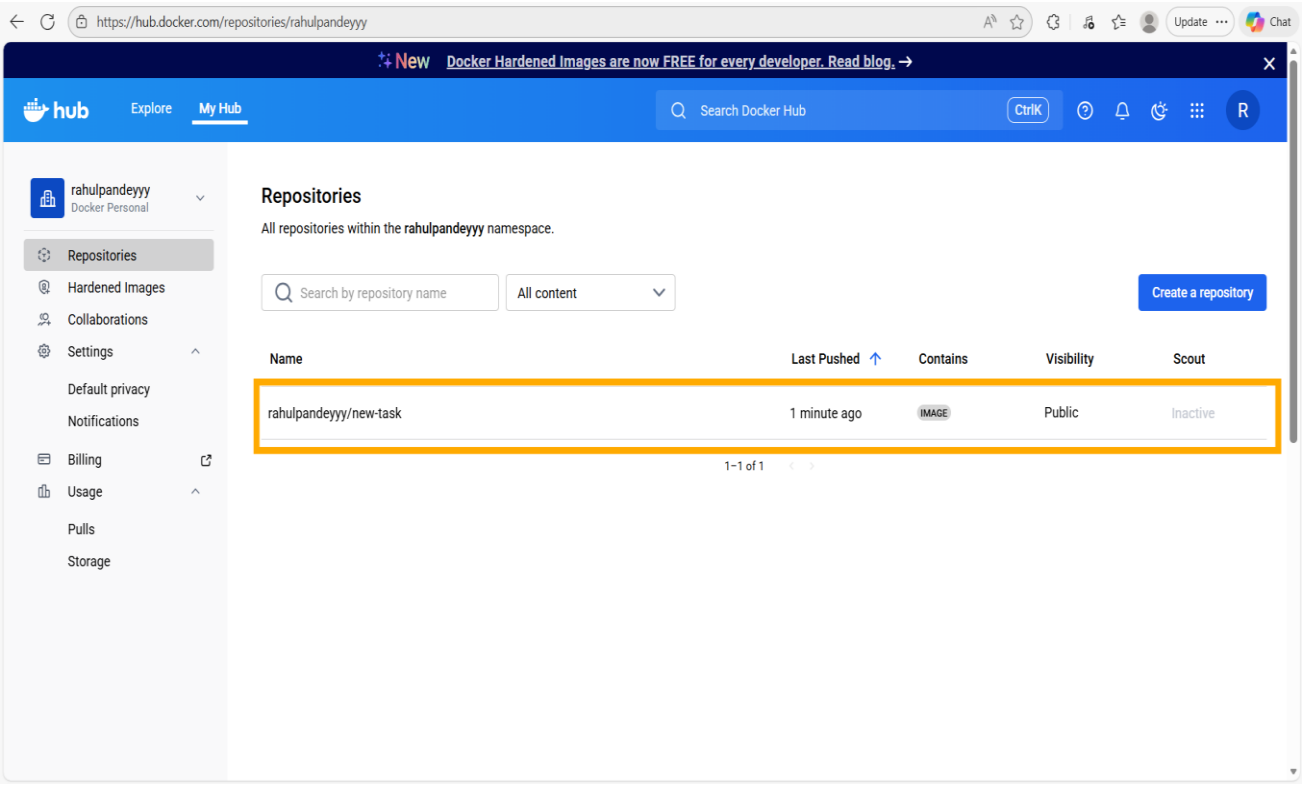
> **Comment:** *The command "sudo docker tag new-task rahulpandeyyy/new-task" creates a new tag for the image under the Docker Hub username "rahulpandeyyy". Running "sudo docker images" now shows three entries: "new-task", "rahulpandeyyy/new-task" (both pointing to the same image ID adf2294e2e93), and "ubuntu". The image is now ready to be pushed to Docker Hub.*

Step 8: Push Image to Docker Hub



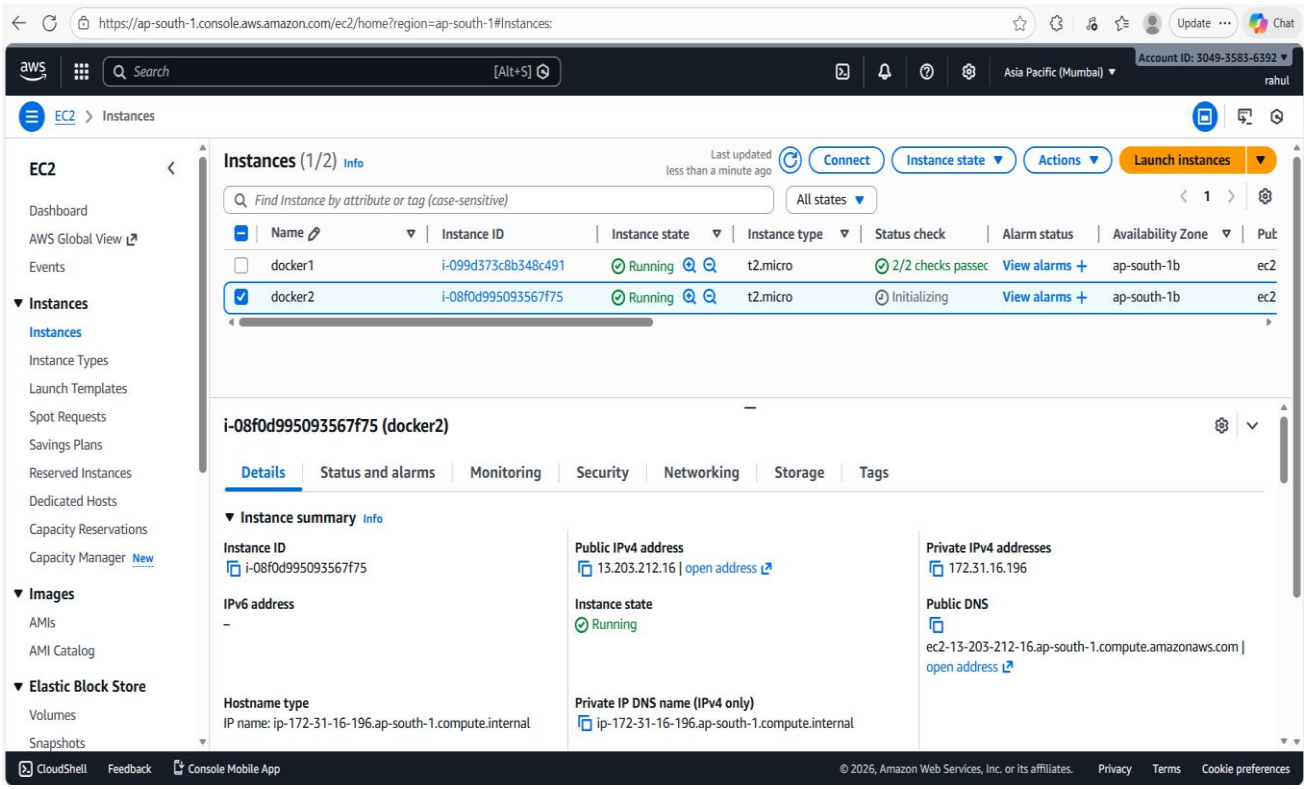
Comment: *"sudo docker push rahulpandeyyy/new-task" pushes the image to Docker Hub. The output shows layer 786bf7bcc0a1 was pushed and layer e8bce0aabd68 was mounted from library/ubuntu. The push completes with digest sha256:a2ee1cdc5fde... and size 741, confirming the image is now publicly available on Docker Hub.*

Step 9: Verify Repository on Docker Hub



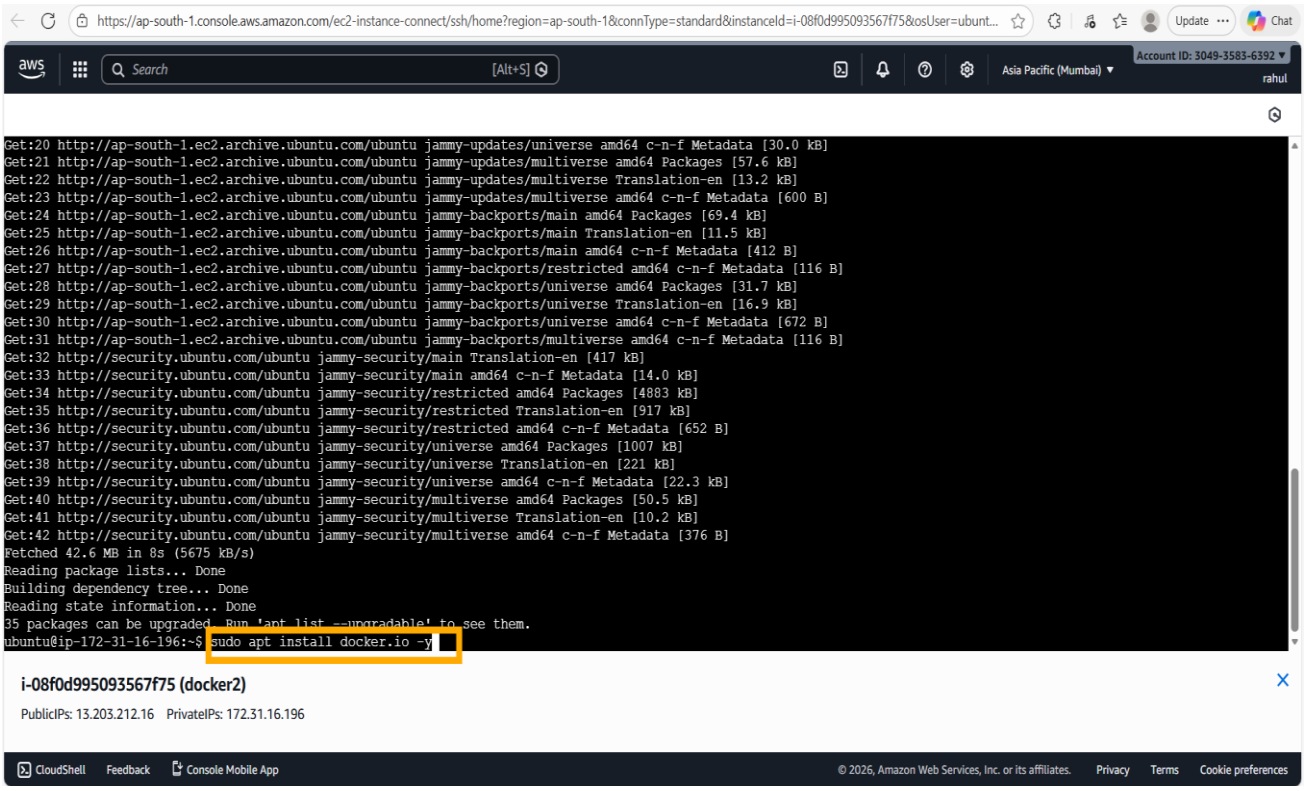
Comment: *The Docker Hub dashboard at https://hub.docker.com/repositories/rahulpandeyyy confirms the "rahulpandeyyy/new-task" repository was created and pushed 1 minute ago. It is visible as a Public image, ready to be pulled by any machine.*

Step 10: Launch Second EC2 Instance (docker2)



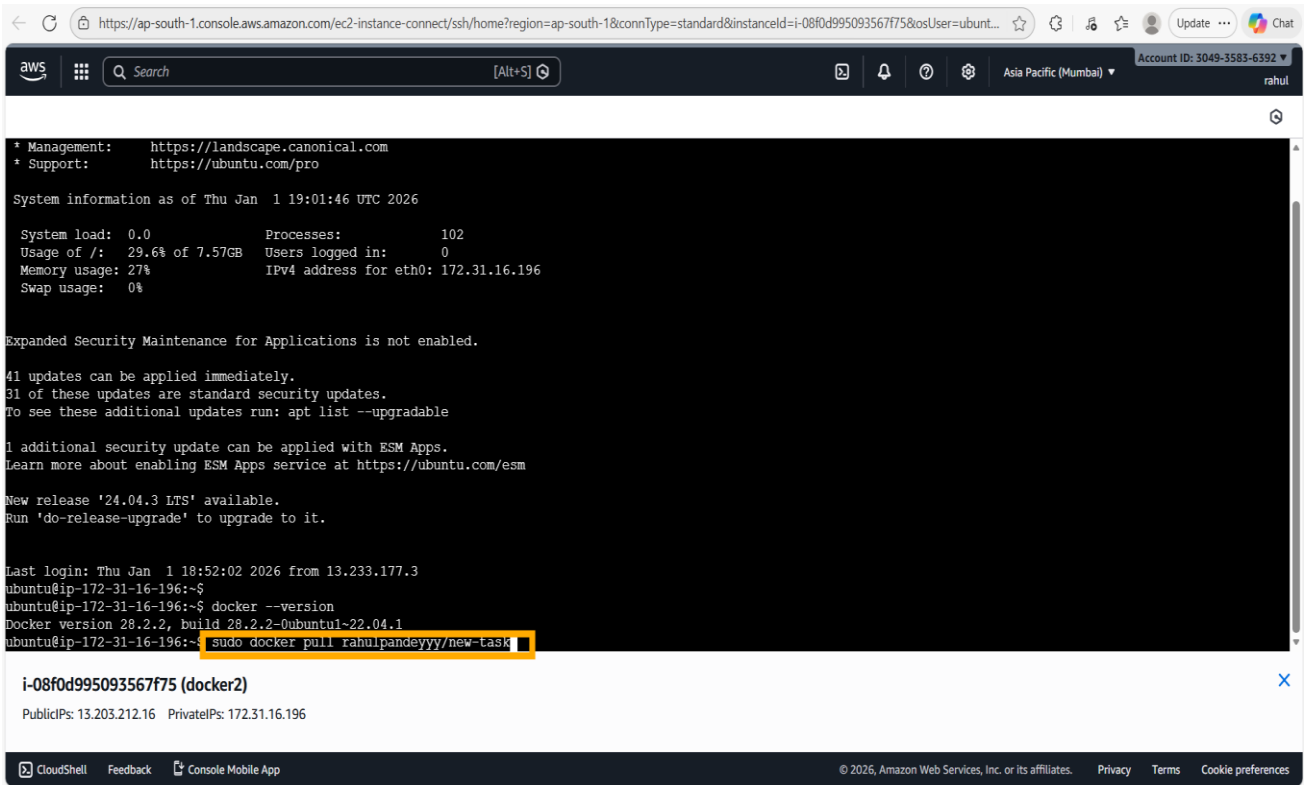
Comment: *A second EC2 instance "docker2" (ID: i-08f0d995093567f75) is launched and shown as "Running" in the AWS console. Its public IPv4 is 13.203.212.16 and private IPv4 is 172.31.16.196. This represents the "separate machine" for pulling and testing the Docker Hub image.*

Step 11: Install Docker on Second Machine



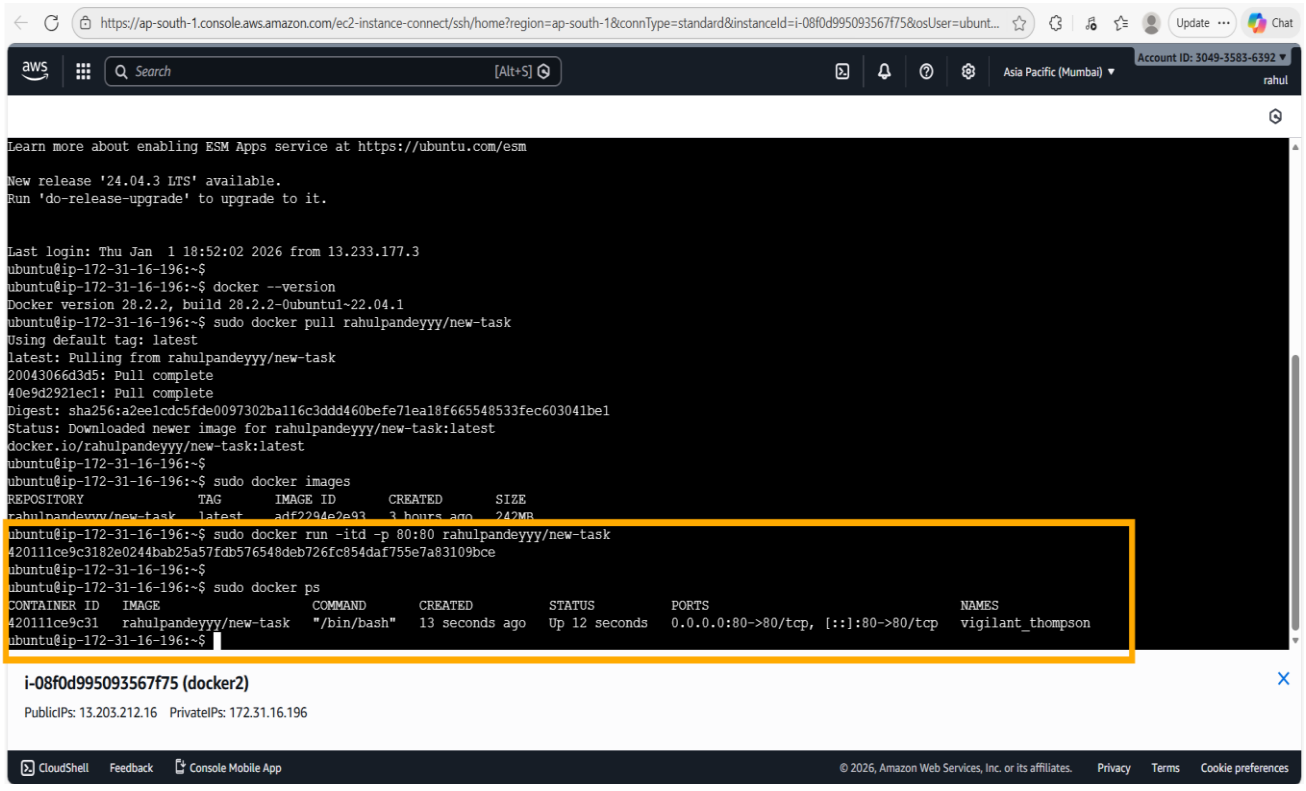
Comment: *On the new "docker2" instance, "sudo apt update" is run first, followed by "sudo apt install docker.io -y" to install Docker. This is needed since this is a fresh EC2 instance with no Docker pre-installed.*

Step 12: Pull Image from Docker Hub on docker2



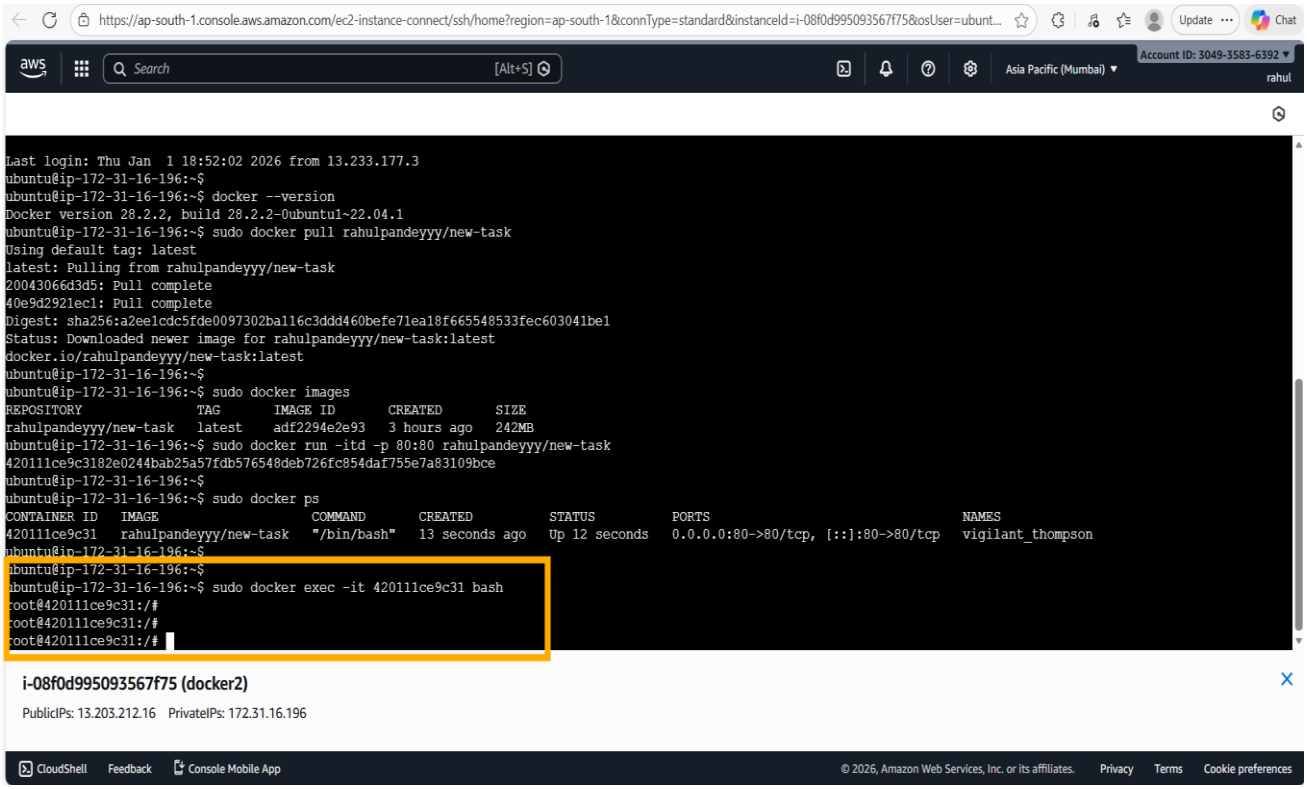
Comment: *On the docker2 instance, Docker version 28.2.2 is confirmed. The command "sudo docker pull rahulpandeyyy/new-task" is executed to pull the image from Docker Hub to this separate machine.*

Step 13: Run the Pulled Container on Port 80



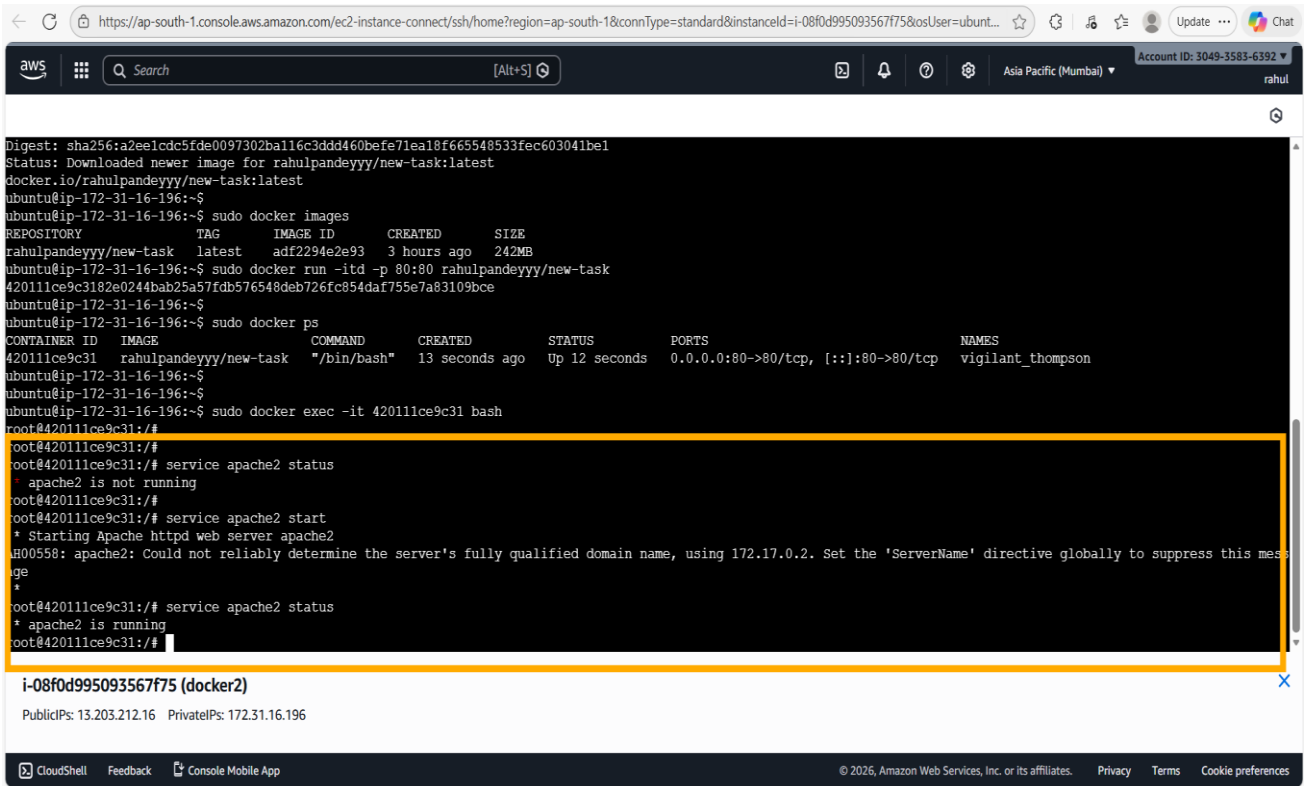
Comment: *The image "rahulpandeyyy/new-task" is pulled successfully (both layers complete). Running "sudo docker run -itd -p 80:80 rahulpandeyyy/new-task" starts a new container (420111ce9c31) mapped on port 80. "sudo docker ps" confirms the container is up and running.*

Step 14: Exec into Container on docker2



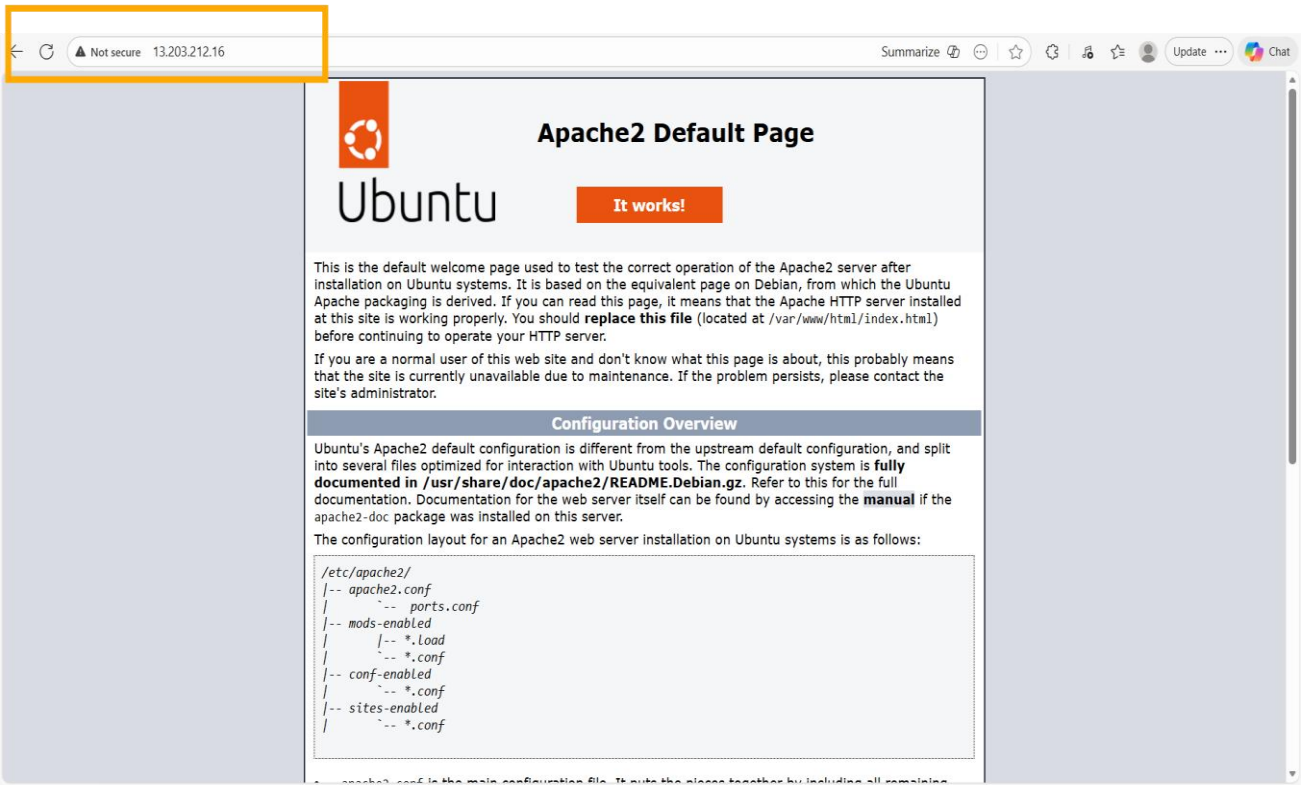
Comment: *"sudo docker exec -it 420111ce9c31 bash" opens a bash shell inside the container on docker2. The shell prompt changes to "root@420111ce9c31:/#", confirming we are inside the container. Apache2 can now be started manually.*

Step 15: Start Apache2 on docker2 Container



Comment: *Inside the container on docker2: "service apache2 status" shows it is not running initially. "service apache2 start" starts Apache2 successfully. A follow-up status check confirms "apache2 is running". The AH00558 ServerName warning is non-critical.*

Step 16: Verify Apache2 on docker2 in Browser



Comment: *The Apache2 default page is accessible at 13.203.212.16 (the docker2 instance's public IP) over port 80. This confirms the complete cross-machine Docker workflow: commit → tag → push to Hub → pull on new machine → run container → start Apache2 → verify in browser.*

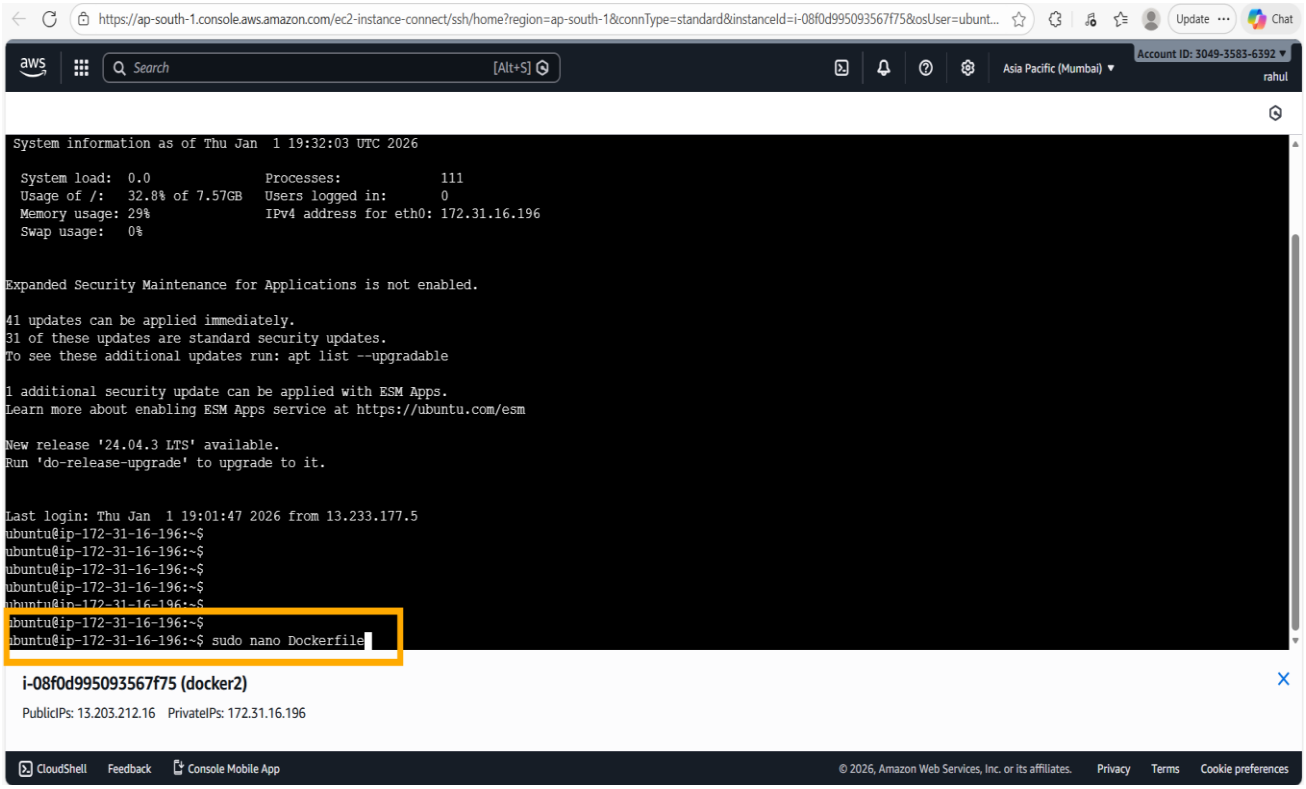
Assignment 4 – Create a Dockerfile for Ubuntu + Apache2

Tasks To Be Performed:

1. Create a Dockerfile with the following specs:
   - Ubuntu base container
   - Apache2 installed
   - Apache2 should automatically run once the container starts
2. Submit the Dockerfile for assignment completion

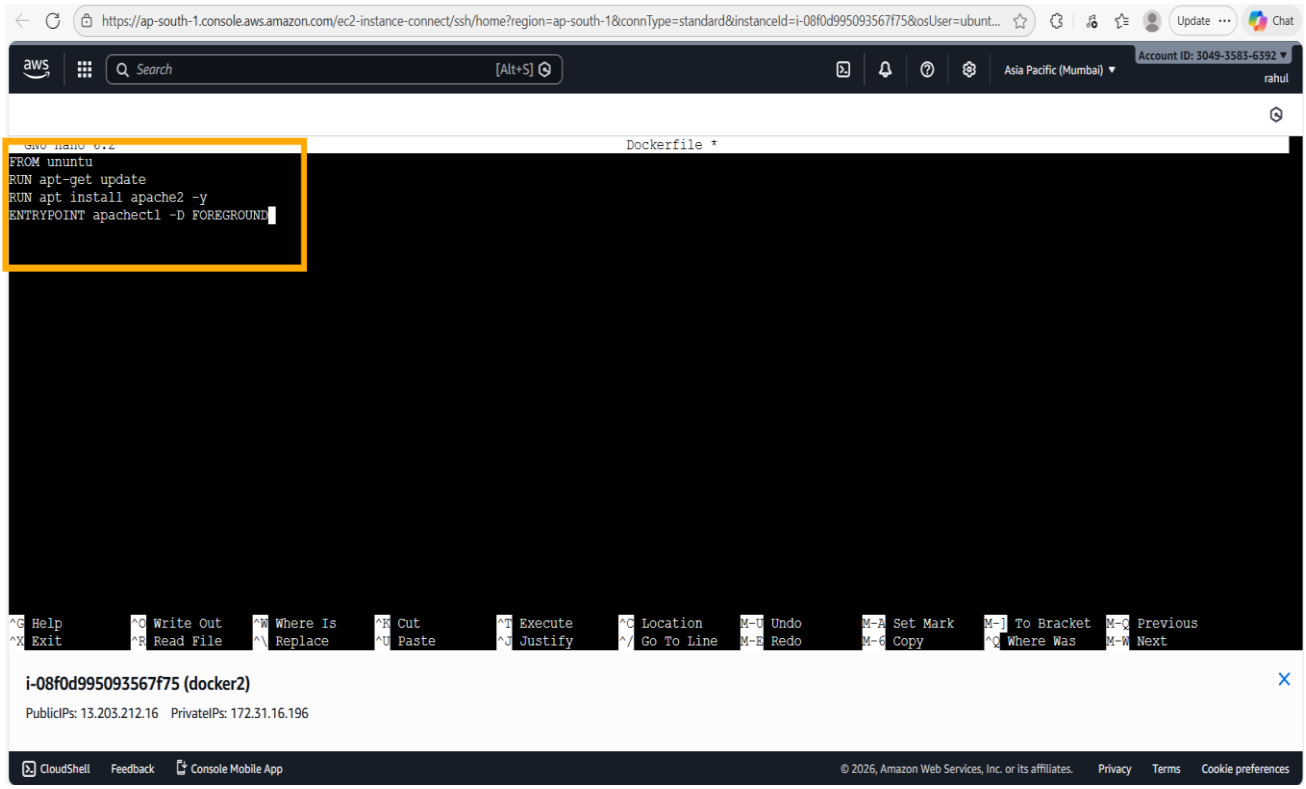
Solution

Step 1: Open Nano to Create Dockerfile



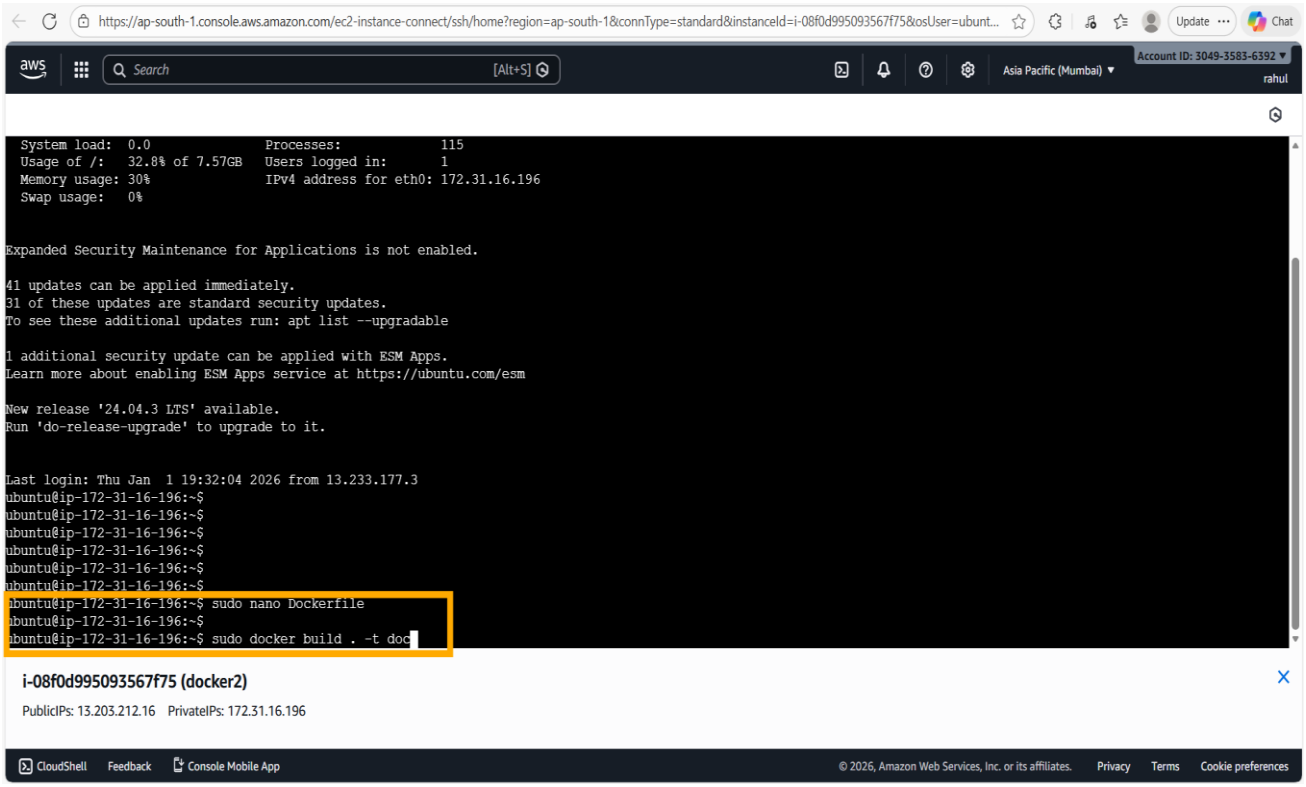
> **Comment:** *On the docker2 instance (172.31.16.196), the command "sudo nano Dockerfile" is used to open the nano text editor and create a new Dockerfile. This is the first step in defining the custom Docker image.*

Step 2: Write the Dockerfile



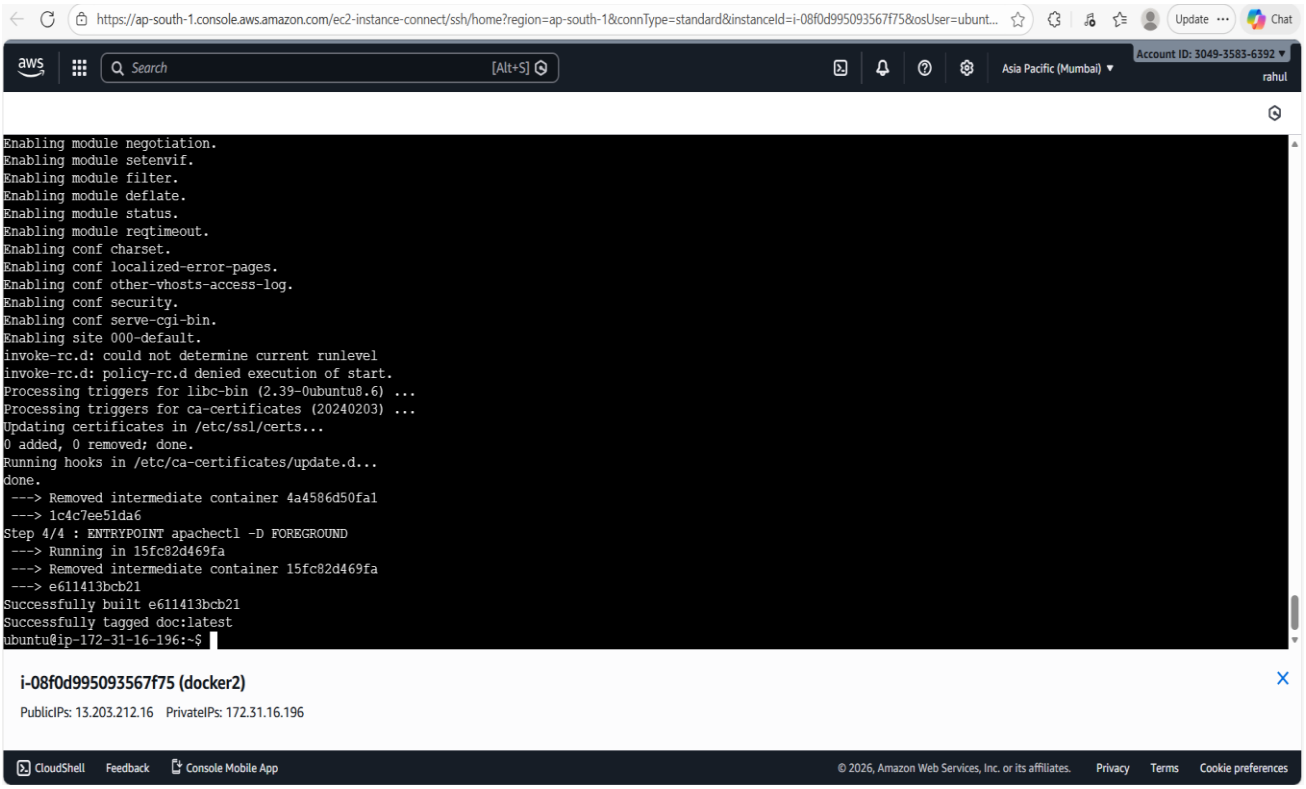
Comment: *The Dockerfile is written with the following four instructions: • FROM ubuntu — uses the official Ubuntu base image • RUN apt-get update — updates package lists during build • RUN apt install apache2 -y — installs Apache2 automatically • ENTRYPOINT apachectl -D FOREGROUND — starts Apache2 in foreground mode so the container stays running. This ensures Apache2 auto-starts whenever the container launches.*

Step 3: Build the Docker Image



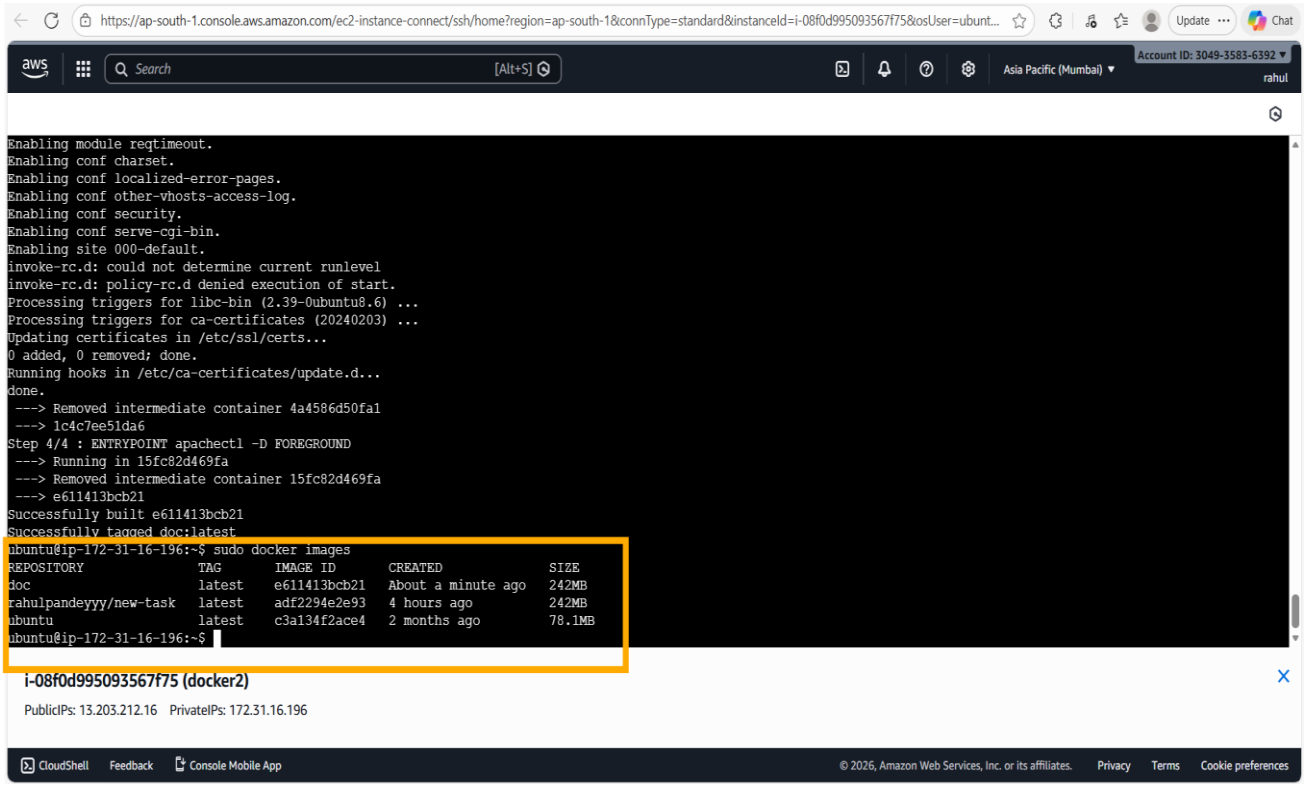
Comment: *After saving the Dockerfile, the command "sudo docker build . -t doc" is executed to build a new Docker image tagged as "doc" from the current directory Dockerfile. The build process will execute all four Dockerfile instructions in sequence.*

Step 4: Docker Build – Steps Execution



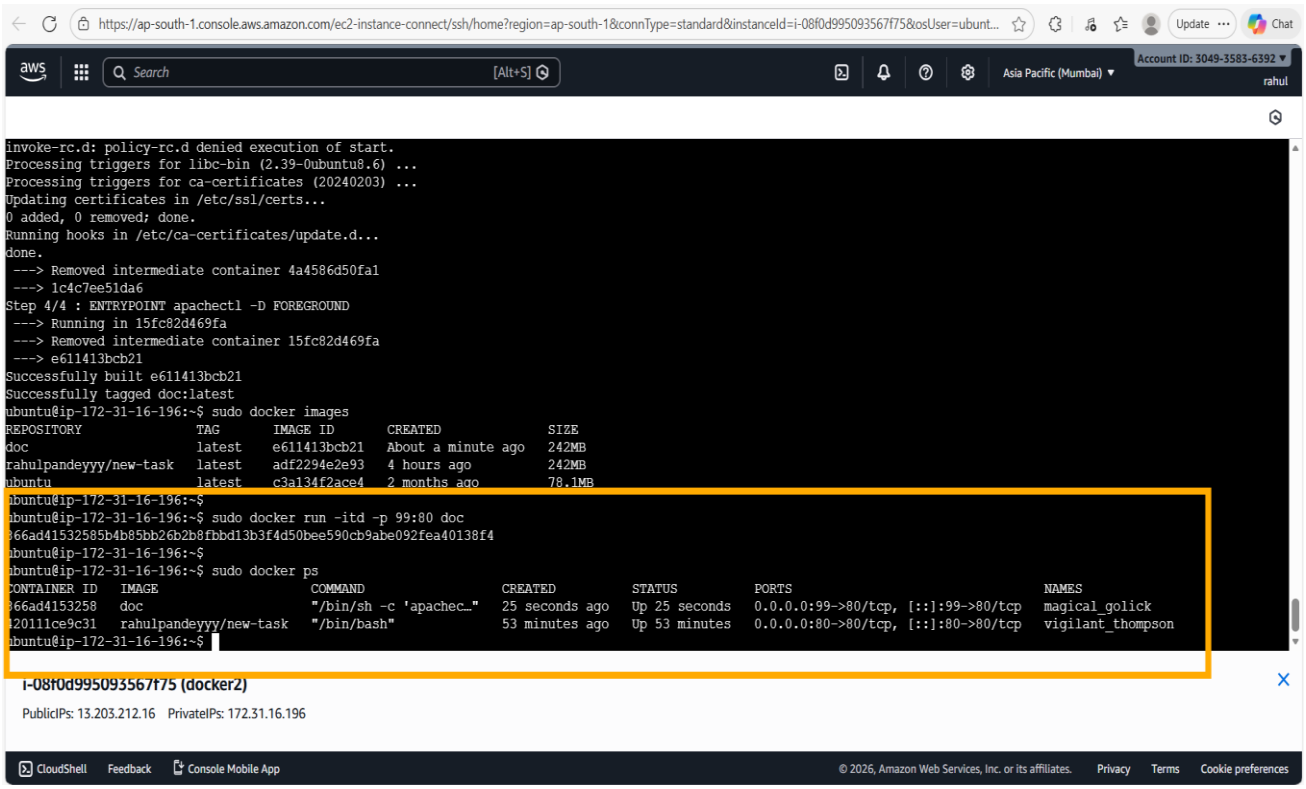
Comment: *The Docker build process completes all 4 steps: • Step 1/4: FROM ubuntu → uses ubuntu base image • Step 2/4: RUN apt-get update → updates packages • Step 3/4: RUN apt install apache2 -y → installs Apache2 (all modules and conf enabled) • Step 4/4: ENTRYPOINT apachectl -D FOREGROUND → sets auto-start entrypoint Build succeeds: "Successfully built e611413bcb21" and "Successfully tagged doc:latest".*

Step 5: Verify New Image & Run Container



Comment: *"sudo docker images" shows the newly built "doc" image (ID: e611413bcb21, 242 MB, created about a minute ago) alongside the existing images. The command "sudo docker run -itd -p 99:80 doc" runs the container from this new image on port 99.*

Step 6: Confirm Running Containers



Comment: *"sudo docker ps" shows both containers running: the "doc" container (866ad4153258) on port 0.0.0.0:99->80/tcp with command "/bin/sh -c 'apachec...'" (confirming the ENTRYPOINT is active), and the previous "rahulpandeyyy/new-task" container on port 80. Notice that the "doc" container automatically started Apache2 via the ENTRYPOINT — no manual "service apache2 start" was needed.*

Step 7: Verify Apache2 Auto-Start in Browser



Comment: *The Apache2 default page is accessible at 13.203.212.16:99 without any manual service start. This confirms that the Dockerfile's ENTRYPOINT instruction "apachectl -D FOREGROUND" successfully auto-starts Apache2 when the container launches, fulfilling the assignment requirement.*

Assignment 5 – Replace Default Apache2 Page with Custom HTML

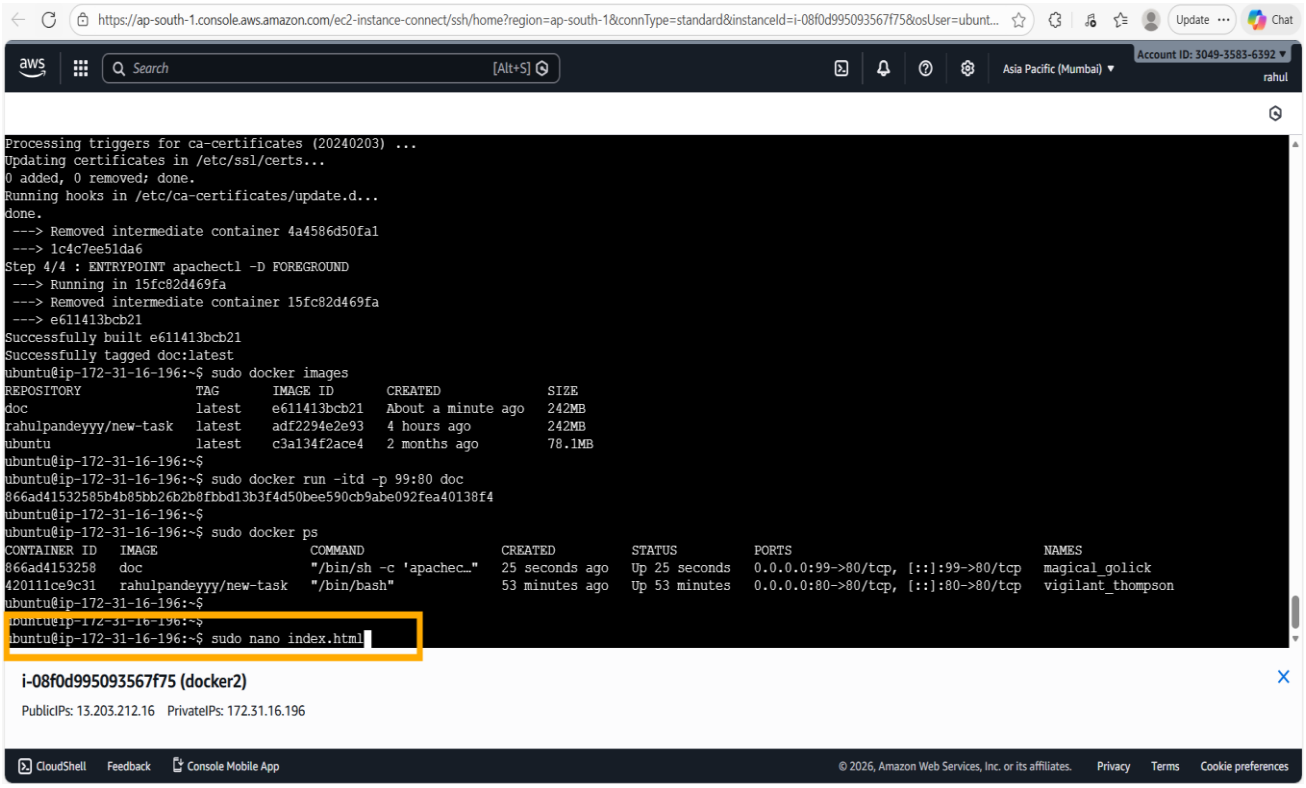
Tasks To Be Performed:

1. Create a sample HTML file
2. Use the Dockerfile from the previous task
3. Replace this sample HTML file inside the Docker container with the default Apache2 page

---

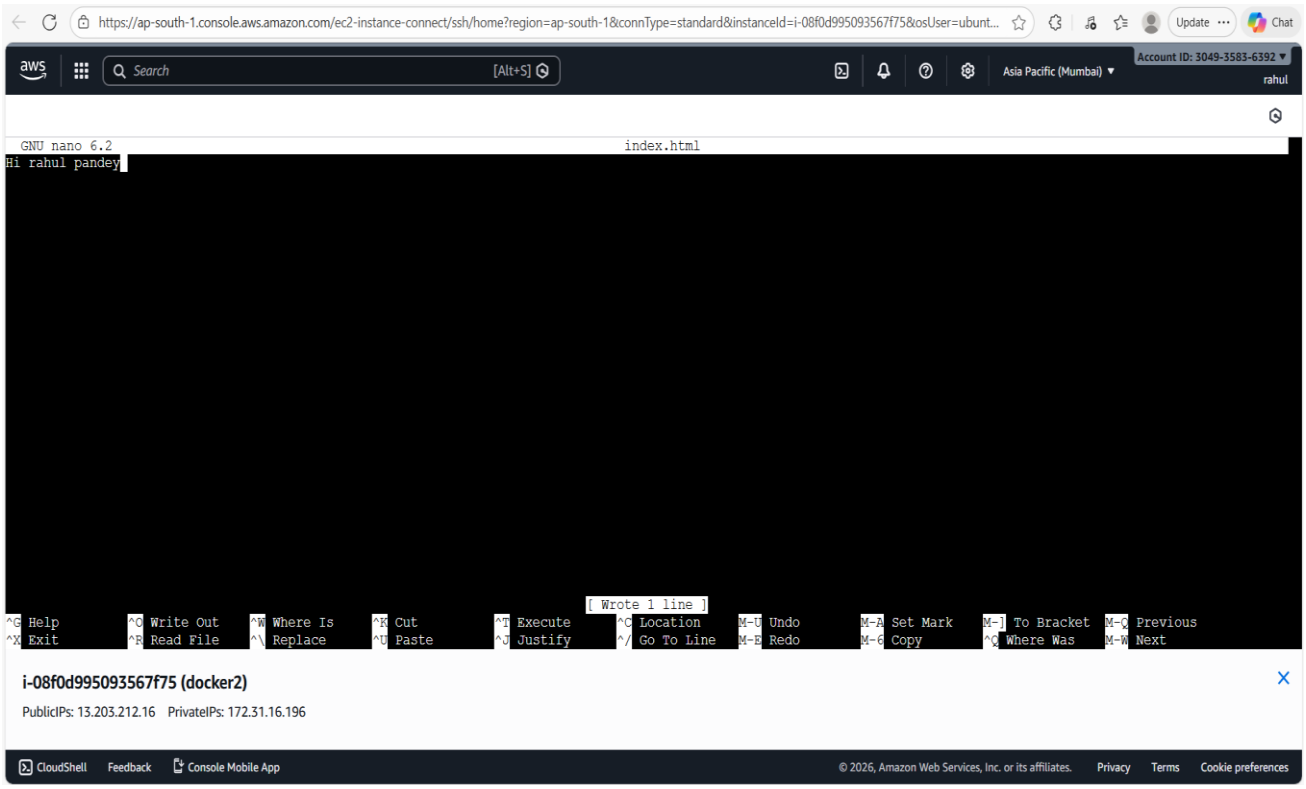
Solution

Step 1: Create index.html and Run work-5 Container



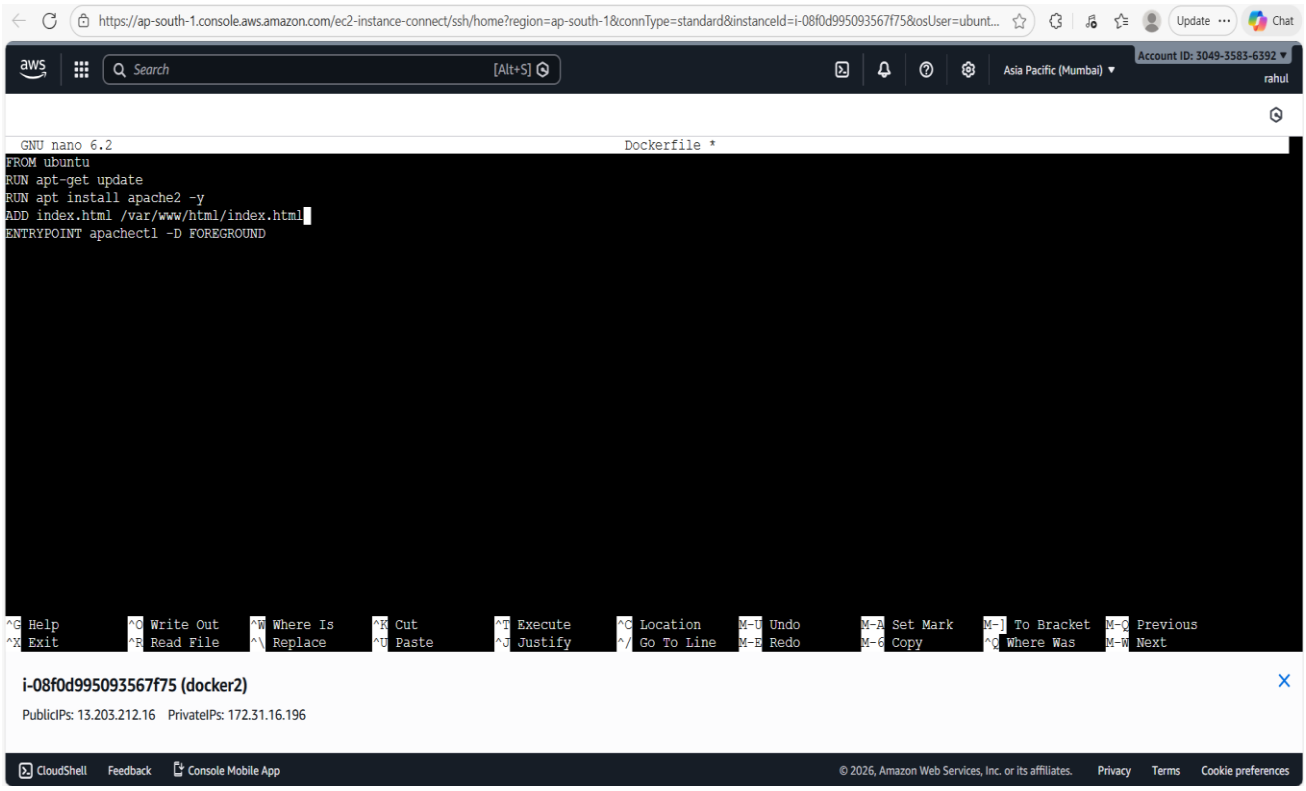
> **Comment:** *After confirming the existing "doc" container on port 99 is running, the command "sudo nano index.html" is used to create a custom HTML file named index.html. This HTML file will be used to replace the default Apache2 page inside the Docker container.*

Step 2: Write Custom HTML Content



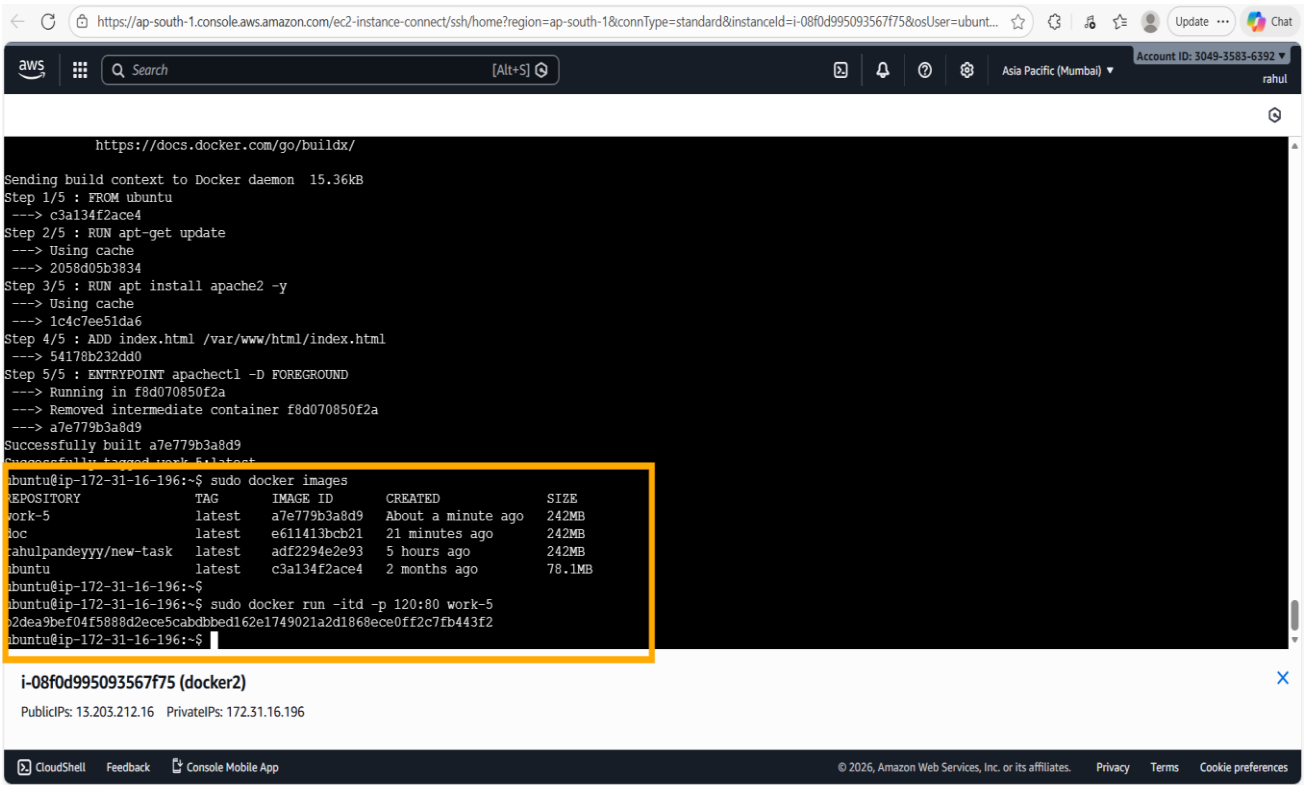
> **Comment:** *The custom HTML file "index.html" is created with the content "Hi rahul pandey" using the nano editor. After saving, "Wrote 1 line" confirms the file was saved successfully. This is the custom web page that will replace Apache2's default index.html.*

Step 3: Update Dockerfile with ADD Instruction



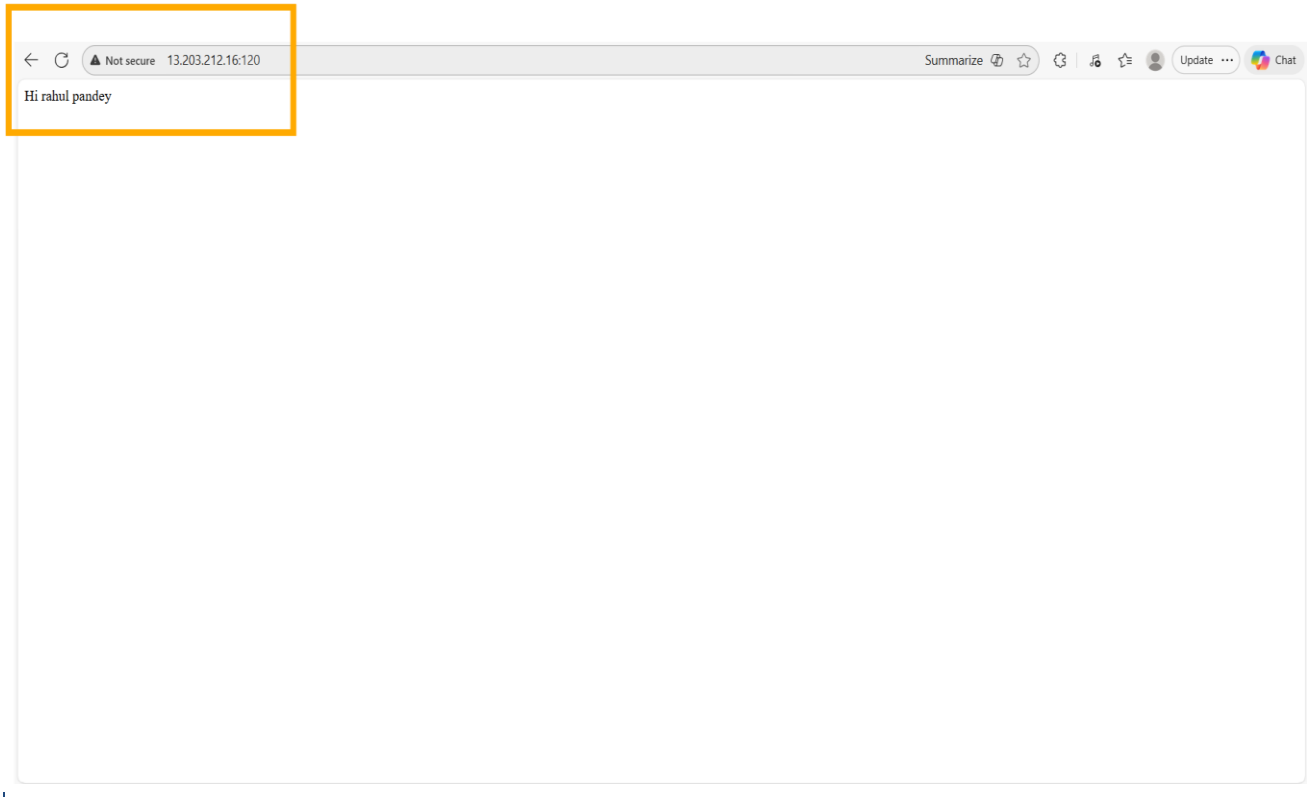
> **Comment:** *The Dockerfile is updated (via "sudo nano Dockerfile") to add a new instruction: • ADD index.html /var/www/html/index.html — this copies the custom index.html from the build context into the container at the Apache2 web root path (/var/www/html/), replacing the default Apache2 welcome page. The complete Dockerfile now has 5 instructions: FROM, RUN apt-get update, RUN apt install, ADD, and ENTRYPOINT.*

Step 4: Build New Image (work-5) & Run on Port 120



> **Comment:** *A new Docker image is built: "sudo docker build . -t work-5". The build output shows 5 steps — Step 4/5 is the new "ADD index.html /var/www/html/index.html" instruction (ID: 54178b232dd0). The image "work-5" (ID: a7e779b3a8d9, 242 MB) is built successfully. Then "sudo docker run -itd -p 120:80 work-5" runs the container on port 120, confirming the new image and container are ready.*

Step 5: Verify Custom HTML Page in Browser



> **Comment:** *The custom HTML page "Hi rahul pandey" is now accessible at 13.203.212.16:120 in the browser. This confirms that the ADD instruction in the Dockerfile successfully replaced the default Apache2 index.html with the custom HTML file at /var/www/html/index.html. The Apache2 server automatically serves the custom page on container start.*